



6장 합성곱 신경망 2 – part 1(~ 6. 1. 4)

1팀 이은수 안서영 조현아

목차

#01 LeNet - 5

#02 AlexNet

#03 VGGNet

#04 GoogLeNet



6. 1. 1 LeNet-5

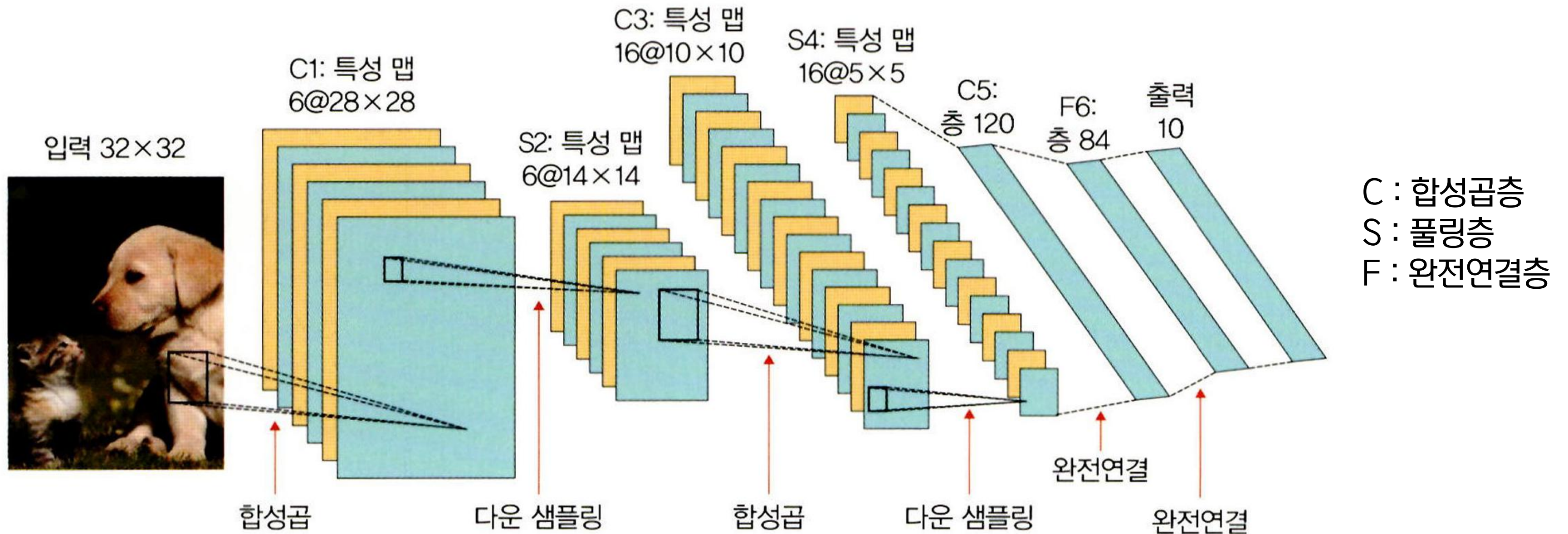


#01 CNN의 시작점, LeNet-5

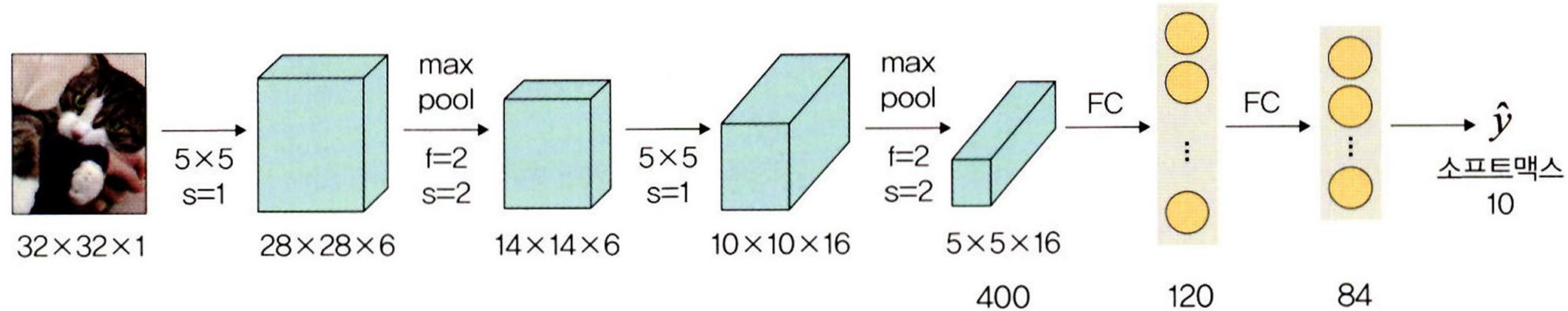
LeNet-5 : 1998년 얀 르쿤 연구팀이 발표한 인공신경망

- 합성곱 신경망(CNN)의 현대적 기본 구조를 정립한 시발점이라는 점에서 딥러닝 역사상 매우 중요한 위치를 차지함
- 합성곱과 다운샘플링(혹은 풀링)을 반복적으로 거치면서 마지막의 완전연결층에서 분류를 수행함

▼ 그림 6-1 LeNet-5



#02 LeNet-5 예제 살펴보기



▼ 표 6-1 LeNet-5 예제 신경망 상세

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	32×32	—	—	—
합성곱층	6	28×28	5×5	1	렐루(ReLU)
최대 풀링층	6	14×14	2×2	2	—
합성곱층	16	10×10	5×5	1	렐루(ReLU)
최대 풀링층	16	5×5	2×2	2	—
완전연결층	—	120	—	—	렐루(ReLU)
완전연결층	—	84	—	—	렐루(ReLU)
완전연결층	—	2	—	—	소프트맥스(softmax)

➔ $32 * 32$ 크기의 이미지에 합성곱층과 최대 풀링층이 쌍으로 두 번 적용된 후 완전연결층을 거쳐 이미지가 분류되는 신경망

#02 LeNet-5 예제 살펴보기

코드 6-1 : 필요한 라이브러리 호출

```
!pip install --user tqdm
import torch
import torchvision
from torch.utils.data import DataLoader, Dataset
from torchvision import transforms # 이미지 변환(전처리) 기능을 제공하는 라이브러리
from torch.autograd import Variable
from torch import optim # 경사하강법을 활용하여 가중치를 구하기 위한 옵티마이저 라이브러리
import torch.nn as nn
import torch.nn.functional as F
import os # 파일 경로에 대한 함수들을 제공
import cv2
from PIL import Image
from tqdm import tqdm_notebook as tqdm # 진행 상황을 가시적으로 표현 - 모델의 학습 경과를 확인하고 싶을 때 사용하는 라이브러리
import random
from matplotlib import pyplot as plt

device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
```

Requirement already satisfied: tqdm in /usr/local/lib/python3.13/dist-packages (4.67.3)

파이토치의 경우 텐서플로와 다르게 GPU를 자동으로 할당하지 않음

→ GPU 할당을 모델과 디바이스에 할당해 주어야 함

→ 교재에서는 CPU를 사용함

#02 LeNet-5 예제 살펴보기

코드 6-2 : 이미지 데이터셋 전처리

```
class ImageTransform():
    def __init__(self, resize, mean, std):
        self.data_transform = {
            'train': transforms.Compose([
                transforms.RandomResizedCrop(resize, scale=(0.5, 1.0)), # 입력 이미지를 주어진 크기로 조정
                transforms.RandomHorizontalFlip(), # 주어진 확률로 이미지를 수평 반전
                transforms.ToTensor(), # torch.FloatTensor 배열로 바꾸어 주는 메서드
                transforms.Normalize(mean, std) # 입력 데이터 정규화 : 데이터의 스케일을 줄이고 0 근처 분포로 만들어 학습을 잘 되게 도움
            ]),
            'val': transforms.Compose([
                transforms.Resize(256),
                transforms.CenterCrop(resize),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ])
        }

    def __call__(self, img, phase):
        return self.data_transform[phase](img)
```

- Train

1. RandomResizedCrop : 무작위 위치 crop, resize

2. RandomHorizontalFlip : 무작위로 좌우 반전

→ Overfitting 방지, 일반화 성능 높임

- val : 무작위 변형 없이 이미지 중앙 잘라냄

→ 모델 성능 측정시 동일하고 공정한 기준으로 평가하기 위함

정규화 : 학습할 때 정규화를 진행 → 모델 가중치 업데이트가 안정적으로 이루어지도록 함

ImageNet 기반 사전 학습 모델을 가져와 전이 학습을 할 때에는 그에 맞는 수치를 똑같이 써야 학습 결과 활용 가능

→ 추후 코드 6-7에서 mean, std와 같은 변수 값을 할당해 주게 됨

#02 LeNet-5 예제 살펴보기

코드 6-3 : 이미지 데이터셋을 불러온 후 훈련, 검증, 테스트로 분리

```
cat_directory = 'dogs-vs-cats/Cat/' # 경로를 합치는 이유
dog_directory = 'dogs-vs-cats/Dog/' : cat_Dirctory에 폴더 경로가 들어 있지만,
                                   폴더 안 개별 사진 파일들의 이름에는 경로가 붙어 있지 않기 때문

cat_images_filepaths = sorted([os.path.join(cat_directory, f) for f in os.listdir(cat_directory)])
# sorted : 데이터를 정렬된 리스트로 만들어서 반환
# os.path.join : 경로와 파일명을 결합 or 분할된 경로를 하나로 합치고 싶을 때 사용
# os.listdir : 지정한 디렉터리 내 모든 파일의 리스트 반환
dog_images_filepaths = sorted([os.path.join(dog_directory, f) for f in os.listdir(dog_directory)])
images_filepaths = [*cat_images_filepaths, *dog_images_filepaths]
correct_images_filepaths = [i for i in images_filepaths if cv2.imread(i) is not None]

random.seed(42) # 특정 시드 값 부여 → 동일한 난수 생성 → 언제 실행해도 똑같은 데이터 분리 결과를 얻음
random.shuffle(correct_images_filepaths)
train_images_filepaths = correct_images_filepaths[:400] # 훈련용 400개 이미지
val_images_filepaths = correct_images_filepaths[400:-10] # 검증용 92개 이미지
test_images_filepaths = correct_images_filepaths[-10:] # 테스트용 10개 이미지
print(len(train_images_filepaths), len(val_images_filepaths), len(test_images_filepaths))
```

400 92 10

#02 LeNet-5 예제 살펴보기

코드 6-4 : 테스트 데이터셋 이미지 확인 검수

```
def display_image_grid(images_filepaths, predicted_labels=(), cols=5):
    rows = len(images_filepaths) // cols
    figure, ax = plt.subplots(nrows=rows, ncols=cols, figsize=(12, 6))
    for i, image_filepath in enumerate(images_filepaths):
        image = cv2.imread(image_filepath)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # 이미지의 색상 변경 : BGR -> RGB
        true_label = os.path.normpath(image_filepath).split(os.sep)[-2] # 경로명 정규화
        predicted_label = predicted_labels[i] if predicted_labels else true_label
        color = "green" if true_label == predicted_label else "red" # 예측과 정답이 동일 - 초록색, 아니면 빨간색
        ax.ravel()[i].imshow(image)
        ax.ravel()[i].set_title(predicted_label, color=color)
        ax.ravel()[i].set_axis_off() # 이미지의 축 제거
    plt.tight_layout() # 이미지의 여백 조정
    plt.show()
```

현재 상황 (학습 전, 검수 단계) : predicted_labels 인자에 아무 값도 들어가지 않음 → 빈 값 상태

→ If 조건문이 False가 됨, 정답이 그대로 대입됨 → 실제 label과 예측 label이 동일, 초록색으로 출력

이후 상황 (학습 후, 실제 평가 단계) : 모델이 실제로 예측한 라벨 리스트를 넘겨 줌

→ 실제 label과 예측 label을 비교하여 맞으면 초록색, 틀리면 빨간색으로 표시됨

#02 LeNet-5 예제 살펴보기

코드 6-5 : 테스트 데이터셋 이미지 출력

```
display_image_grid(test_images_filepaths)
```

Cat



Dog



Cat



Cat



Cat



Cat



Dog



Cat



Cat



Dog



#02 LeNet-5 예제 살펴보기

코드 6-6 : 이미지 데이터셋 클래스 정의

```
class DogvsCatDataset(Dataset):
    def __init__(self, file_list, transform=None, phase='train'): # 데이터셋의 전처리
        self.file_list = file_list
        self.transform = transform
        self.phase = phase

    def __len__(self):
        return len(self.file_list)

    def __getitem__(self, idx):
        img_path = self.file_list[idx]
        img = Image.open(img_path) # img_path 위치에서 이미지 데이터들을 가져옴
        img_transformed = self.transform(img, self.phase) # 이미지에 'train' 전처리를 적용

        label = img_path.split('/')[-1].split('.')[0] # / 와 . 를 제거, 분리된 값에서 index 0를 가져옴
        if label == 'dog':
            label = 1
        elif label == 'cat':
            label = 0
        return img_transformed, label
```

#고양이가 있는 이미지의 레이블 → 0

개가 있는 이미지의 레이블 → 1

코드 6-7 : 변수 값 정의

```
size = 224
mean = (0.485, 0.456, 0.406)
std = (0.229, 0.224, 0.225)
batch_size = 32
```

전처리에 사용할 변수에 대한 값을 정의

#02 LeNet-5 예제 살펴보기

코드 6-8 : 이미지 데이터셋 정의

```
train_dataset = DogvsCatDataset(train_images_filepaths, transform=ImageTransform(size, mean, std), phase='train')
# 훈련 데이터에 train_transforms를 적용
val_dataset = DogvsCatDataset(val_images_filepaths, transform=ImageTransform(size, mean, std), phase='val')
# 검증 이미지에 train_transforms를 적용
```

```
index = 0 # 첫번째 이미지 지정 (맨 첫번째 데이터 하나만 꺼내옴)
print(train_dataset.__getitem__(index)[0].size()) # 훈련 데이터의 크기 출력
print(train_dataset.__getitem__(index)[1]) # 훈련 데이터의 레이블 출력
```

```
torch.Size([3, 224, 224])
0
```

훈련과 검증 용도의 데이터셋 정의 → 전처리도 함께 적용

훈련 데이터의 크기와 레이블 출력

→ 첫 번째 이미지의 전처리된 텐서 데이터의 크기가 [3, 224, 224] : 3채널, 224*224 크기

→ 첫 번째 이미지의 클래스 라벨 숫자가 0 : '고양이' 라는 이미지가 포함되어 있음을 유추해볼 수 있음

#02 LeNet-5 예제 살펴보기

코드 6-9 : 데이터로더 정의

```
train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True) # 앞 코드 6-7에서 설정한 배치 사이즈(32)
val_dataloader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False) # 데이터를 조금씩 쪼개어 가져옴
dataloader_dict = {'train': train_dataloader, 'val': val_dataloader}
# 훈련 데이터셋과 검증 데이터셋을 합쳐서 표현
```

```
batch_iterator = iter(train_dataloader)
inputs, label = next(batch_iterator)
print(inputs.size())
print(label)
```

```
torch.Size([32, 3, 224, 224])
tensor([0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 0, 0, 0, 0,
        1, 1, 1, 1, 1, 1, 1, 0])
```

훈련 데이터셋과 검증 데이터셋의 shuffle 옵션

- 훈련 데이터셋 : shuffle = True

- Overfitting 및 bias 방지 : 매 epoch마다 데이터 순서를 섞어 주어야 특정 순서 패턴이 아닌 이미지의 특징을 학습

- 학습 안정화

- 검증 데이터셋 : shuffle = False

- 평가 기준의 일관성, 순서 불변 : 모델의 가중치를 업데이트 하는 단계가 아니라 모델의 실력을 평가만 하는 단계

- 데이터를 섞든 안 섞든 전체 정확도 및 loss 결과는 동일함

#02 LeNet-5 예제 살펴보기

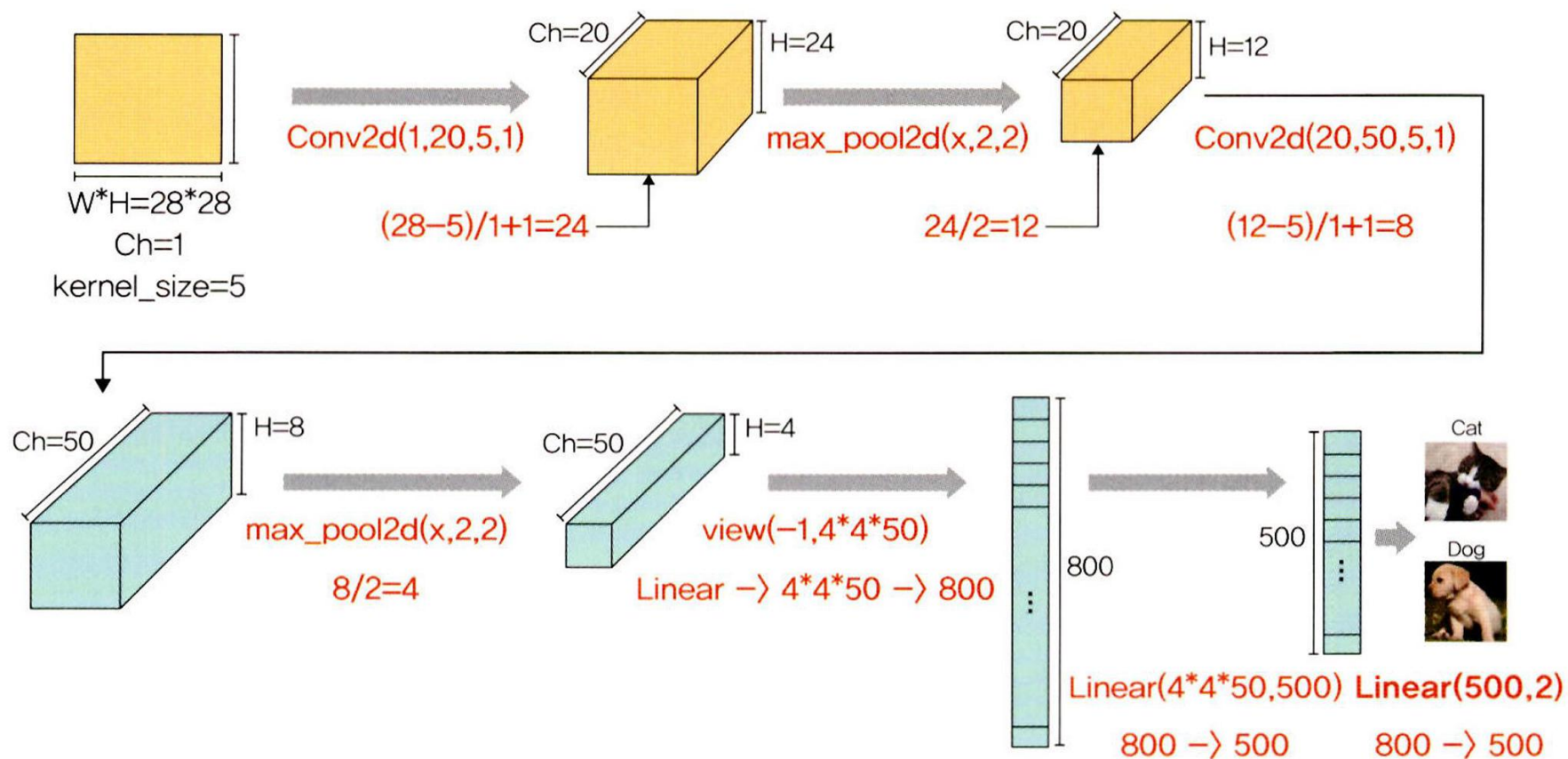
```
class LeNet(nn.Module):
    def __init__(self):
        super(LeNet, self).__init__()
        self.cnn1 = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=5, stride=1, padding=0) #2d 합성곱층 적용
        self.relu1 = nn.ReLU() # 활성화 함수
        self.maxpool1 = nn.MaxPool2d(kernel_size=2) # 최대 풀링 적용
        self.cnn2 = nn.Conv2d(in_channels=16, out_channels=32, kernel_size=5, stride=1, padding=0) #2d 합성곱층 적용
        self.relu2 = nn.ReLU() # 활성화 함수
        self.maxpool2 = nn.MaxPool2d(kernel_size=2) # 최대 풀링 적용
        self.fc1 = nn.Linear(32*5*5, 512) # 완전 연결층
        self.relu5 = nn.ReLU() # 활성화 함수
        self.fc2 = nn.Linear(512, 2) # 완전 연결층 : 최종 출력 노드 수를 2로 설정 - 고양이 / 개 판단
        self.output = nn.Softmax(dim=1) # 최종 출력값을 2개 클래스에 대한 확률 값으로 변환
```

Conv2d를 거쳤을 때의 출력 크기 공식

$$\text{출력 크기} = \frac{\text{입력 크기} - \text{커널 크기}}{\text{보폭(Stride)}} + 1$$

```
def forward(self, x):
    out = self.cnn1(x)
    out = self.relu1(out)
    out = self.maxpool1(out)
    out = self.cnn2(out)
    out = self.relu2(out)
    out = self.maxpool2(out)
    out = out.view(out.size(0), -1)
    out = self.fc1(out)
    out = self.relu5(out)
    out = self.fc2(out)
    out = self.output(out)
    return out
```

▼ 그림 6-5 합성곱층과 풀링층의 크기(형태) 계산



→ 28*28 크기의 이미지 1장이 연산을 거치며 50*4*4 크기로 축소되고, 이를 1차원으로 펼쳐 총 800개의 특징값을 추출한 뒤 2개의 label로 최종 분류

#02 LeNet-5 예제 살펴보기

코드 6-11 : 모델 객체 생성

```
model = LeNet().to(device)
print(model)
```

숨겨진 출력 표시

코드 6-12 : torchsummary 라이브러리를 이용한 모델의 네트워크 구조 확인

```
!pip install torchsummary
from torchsummary import summary
summary(model, (3, 224, 224))
```

Requirement already satisfied: torchsummary in /usr/local/lib/python3.13/dist-packages (1.5.1)

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 16, 220, 220]	1,216
ReLU-2	[-1, 16, 220, 220]	0
MaxPool2d-3	[-1, 16, 110, 110]	0
Conv2d-4	[-1, 32, 106, 106]	12,832
ReLU-5	[-1, 32, 106, 106]	0
MaxPool2d-6	[-1, 32, 53, 53]	0
Linear-7	[-1, 512]	46,023,168
Linear-8	[-1, 2]	1,026
Softmax-9	[-1, 2]	0

```
Total params: 46,038,242
Trainable params: 46,038,242
Non-trainable params: 0
```

```
LeNet(
  (cnn1): Conv2d(3, 16, kernel_size=(5, 5), stride=(1, 1))
  (relu1): ReLU()
  (maxpool1): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (cnn2): Conv2d(16, 32, kernel_size=(5, 5), stride=(1, 1))
  (relu2): ReLU()
  (maxpool2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (fc1): Linear(in_features=89888, out_features=512, bias=True)
  (relu5): ReLU()
  (fc2): Linear(in_features=512, out_features=2, bias=True)
  (output): Softmax(dim=1)
)
```

torchsummary 라이브러리 활용

→ 모델의 네트워크 정보를 한눈에 파악할 수 있음

→ 총 파라미터 수, 입력 크기, 네트워크의 총 크기 등

#02 LeNet-5 예제 살펴보기

코드 6-14 : 옵티마이저와 손실 함수 정의

```
optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9) # 경사 하강법 - 모멘텀 SGD 사용
criterion = nn.CrossEntropyLoss() # 손실함수 정의
```

코드 6-15 : 모델의 파라미터와 손실 함수를 CPU에 할당
GPU를 활성화하지 않은 상태 - CPU 사용

```
model = model.to(device)
criterion = criterion.to(device)
```

옵티마이저 설정 : 경사하강법 - 모멘텀 SGD 활용

- 모멘텀 SGD : SGD에 관성이 추가된 것

→ 가중치를 수정하기 전 이전 수정 방향을 참고하여 같은 방향으로 일정한 비율만 수정되도록 함

- Lr : 학습률 - 가중치 변경시 얼마나 크게 변경할 것인지

- Momentum : 관성 - 흔들림(진동)을 줄여 주는 매개 변수

GPU가 활성화되어 있지 않기 때문에 CPU 사용

#02 LeNet-5 예제 살펴보기

코드 6-16 : 모델 학습 함수 정의

```
def train_model(model, dataloader_dict, criterion, optimizer, num_epoch):
    since = time.time()
    best_acc = 0.0

    for epoch in range(num_epoch):
        print('Epoch {}/{}'.format(epoch + 1, num_epoch))
        print('-'*20)

        for phase in ['train', 'val']:
            if phase == 'train':
                model.train() # 모델을 학습시키겠다는 의미
            else:
                model.eval()

            epoch_loss = 0.0
            epoch_corrects = 0

            for inputs, labels in tqdm(dataloader_dict[phase]):
                inputs = inputs.to(device) # 훈련 데이터셋을 CPU에 할당
                labels = labels.to(device)
                optimizer.zero_grad() # 역전파 실행 전 기울기를 0으로 초기화

                with torch.set_grad_enabled(phase == 'train'):
                    outputs = model(inputs)
                    _, preds = torch.max(outputs, 1)
                    loss = criterion(outputs, labels) # loss 계산

                if phase == 'train':
                    loss.backward() # 학습 가능한 파라미터에 대해 기울기 계산
                    optimizer.step() # 파라미터 갱신
```

```
            epoch_loss += loss.item() * inputs.size(0) # 오차와 입력을 곱함
            epoch_corrects += torch.sum(preds == labels.data) # 예측과 정답이 일치하면 epoch_corrects에 저장

            epoch_loss = epoch_loss / len(dataloader_dict[phase].dataset) # 최종 오차 계산
            epoch_acc = epoch_corrects.double() / len(dataloader_dict[phase].dataset) # 최종 정확도 계산

            print('{} Loss: {:.4f} Acc: {:.4f}'.format(phase, epoch_loss, epoch_acc))

            if phase == 'val' and epoch_acc > best_acc: # 검증 데이터셋에 대한 가장 최적의 정확도를 저장
                best_acc = epoch_acc
                best_model_wts = model.state_dict()

        time_elapsed = time.time() - since
        print('Training complete in {:.0f}m {:.0f}s'.format(
            time_elapsed // 60, time_elapsed % 60))
        print('Best val Acc: {:.4f}'.format(best_acc))
    return model
```

loss.item()과 inputs.size(0)을 곱하는 이유?

- 파이토치의 손실함수(ex.nn.CrossEntropyLoss)는 reduction 인자 기본 값이 'mean' 으로 설정되어 있음
- 데이터 각각에 대해 오차를 계산 후 input size로 나눈 평균값 하나(loss.item())을 반환함
- 에포크 전체의 평균 손실을 구하기 위해서는 전체 데이터의 총 손실합을 구한 뒤 마지막에 전체 데이터 수로 나눠야 함

→ 오차 평균에 데이터 개수를 곱하여 전체 데이터의 총 손실합을 구하게 됨

EWCHA
EUROPEAN

→ 더 정확한 결과를 얻기 위해서는 데이터셋을 늘려볼 수 있음

#02 LeNet-5 예제 살펴보기

코드 6-18 : 모델 테스트를 위한 함수 정의

```
import pandas as pd

id_list = []
pred_list = []
_id=0
with torch.no_grad(): # 역전파 중 텐서들에 대한 변화도를 계산할 필요가 없음 -> 훈련 모델과의 가장 큰 차이점
    for test_path in tqdm(test_images_filepaths): # 테스트 데이터셋 이용
        img = Image.open(test_path)
        _id = test_path.split('/')[-1].split('.')[1]
        transform = ImageTransform(size, mean, std)
        img = transform(img, phase='val') # 테스트 데이터셋 전처리 적용
        img = img.unsqueeze(0) # 텐서에 차원 추가, 0 : 차원이 추가될 위치
        img = img.to(device)

        model.eval()
        outputs = model(img)
        preds = F.softmax(outputs, dim=1)[0, 1].tolist() # 텐서의 요소가 (0,1) 범위에 있고 합계가 1이 되도록 크기 조정
        id_list.append(_id)
        pred_list.append(preds[0])

res = pd.DataFrame({
    'id': id_list,
    'label': pred_list
}) # 테스트 데이터셋의 예측 결과인 id와 label을 데이터 프레임에 저장

res.sort_values(by='id', inplace=True)
res.reset_index(drop=True, inplace=True)

res.to_csv('LesNet.csv', index=False) # 데이터 프레임을 csv 파일로 저장
```

차원을 추가하는 이유

- 테스트 코드의 경우 DataLoader를 거치지 않고 Image.open()으로 이미지를 1장씩 읽어서 전처리함
→ 이 상태 그대로 모델에 넘겨주면 차원 불일치 error가 발생

- 맨 앞에 배치 차원 1을 덧붙여 주어 "지금 1장짜리 배치 데이터 1묶음이 들어간다"고 알려주는 것
- img.unsqueeze(0) 실행 전: [3, 224, 224]
- img.unsqueeze(0) 실행 후: [1, 3, 224, 224]
(0번째 자리에 크기 1인 차원 추가)

#02 LeNet-5 예제 살펴보기

코드 6-19 : 테스트 데이터셋의 예측 결과 호출

```
res.head(10)
```

	id	label
0	109	0.514580
1	145	0.409990
2	15	0.608952
3	162	0.536567
4	167	0.523500
5	200	0.498173
6	210	0.630094
7	211	0.581465
8	213	0.418457
9	224	0.597991

예측 결과 label > 0.5 → 개
예측 결과 label < 0.5 → 고양이

코드 6-20 : 테스트 데이터셋 이미지를 출력하기 위한 함수 정의

```
class_ = classes = {0: 'cat', 1: 'dog'} # 개와 고양이에 대한 클래스 정의
def display_image_grid(images_filepaths, predicted_labels=(), cols=5):
    rows = len(images_filepaths) // cols
    figure, ax = plt.subplots(nrows=rows, ncols=cols, figsize=(12, 6))
    for i, image_filepath in enumerate(images_filepaths):
        image = cv2.imread(image_filepath)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        a = random.choice(res['id'].values)
        label = res.loc[res['id'] == a, 'label'].values[0]
        if label > 0.5: # 레이블 값이 0.5보다 크다면 개
            label = 1
        else: # 작다면 고양이
            label = 0
        ax.ravel()[i].imshow(image)
        ax.ravel()[i].set_title(class_[label])
        ax.ravel()[i].set_axis_off()
    plt.tight_layout()
    plt.show()
```

#02 LeNet-5 예제 살펴보기

코드 6-21 : 테스트 데이터셋 예측 결과 이미지 출력

`display_image_grid(test_images_filepaths)`

극히 일부의 데이터를 이용한 모델 학습 → 예측력이 좋지 않음

→ 성능 개선 방안 : 캐글에서 내려받은 데이터 전체 활용

dog



cat



dog



dog



cat



cat



dog



dog



cat



cat



6.1.2 AlexNet



AlexNet 개요

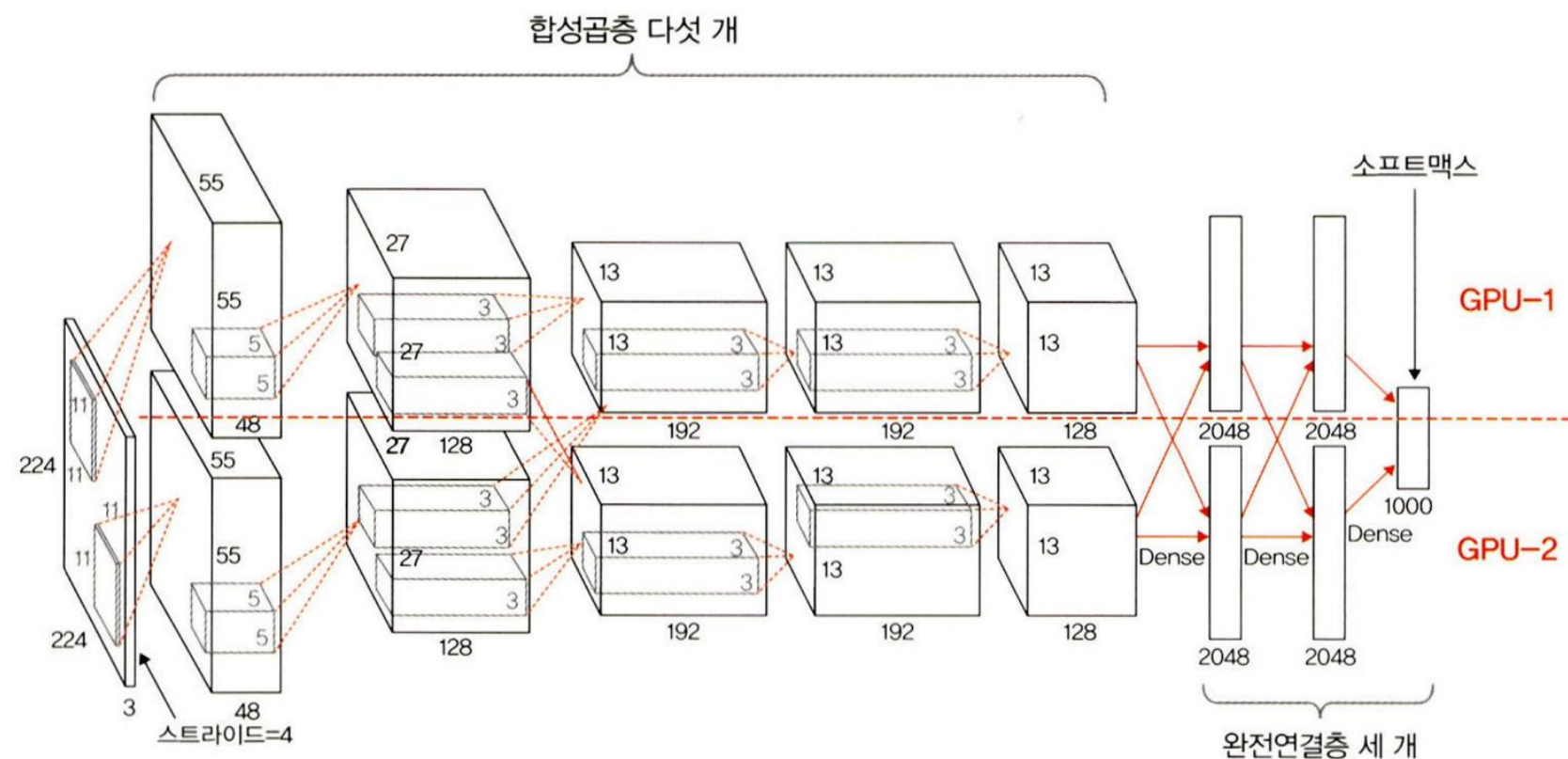
: LeNet-5의 CNN 구조를 크게 확장하여, 대규모 이미지 분류를 가능하게 한 대표적인 CNN 구조

CNN 복습

- 3차원 구조(이미지를 다루기 때문)
- **깊이(depth)**를 가짐
 - 합성곱을 거치며 특성맵이 만들어지고 이에 따라 중간 영상의 깊이가 달라짐

AlexNet 구조

- ① GPU 두 개를 기반으로 한 병렬 구조
- ② 합성곱층 5개 + 완전연결층 3개



AlexNet 개요

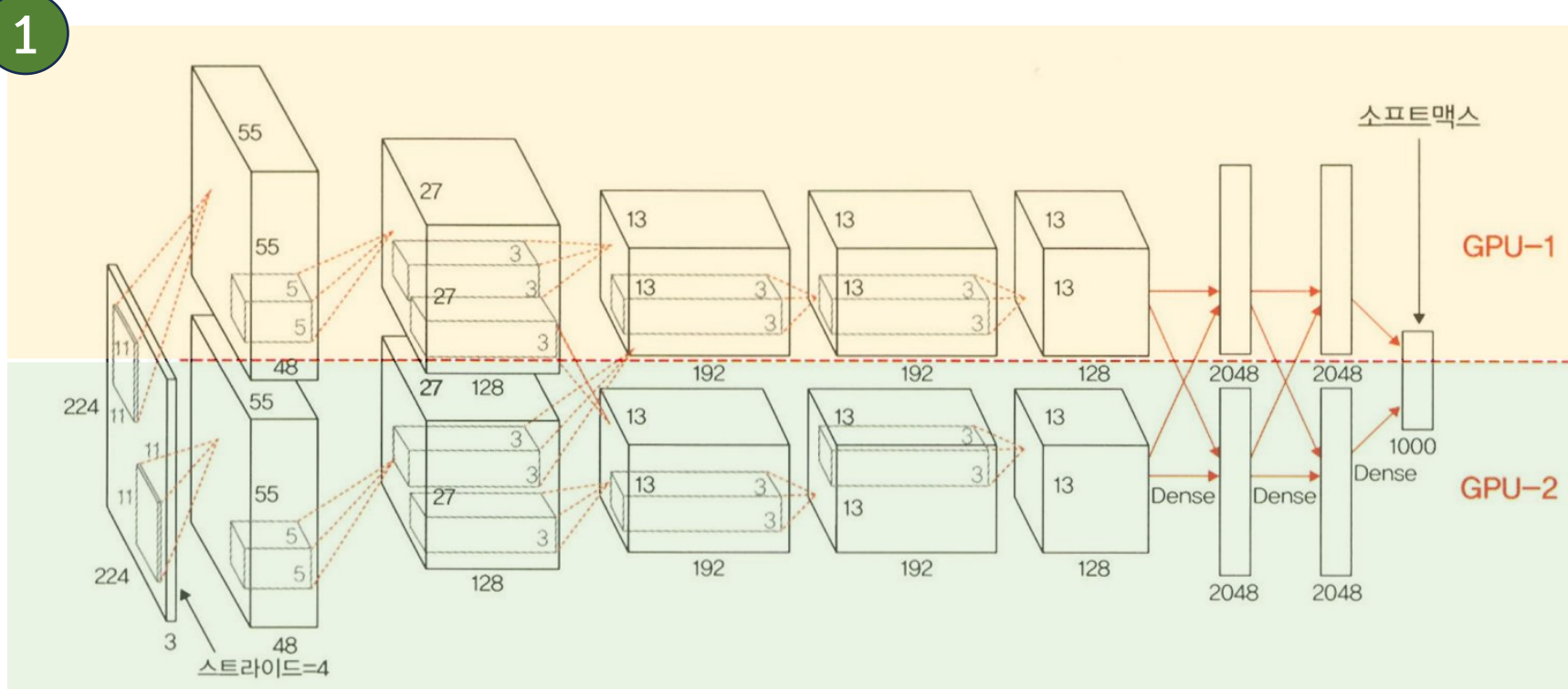
: LeNet-5의 CNN 구조를 크게 확장하여, 대규모 이미지 분류를 가능하게 한 대표적인 CNN 구조

CNN 복습

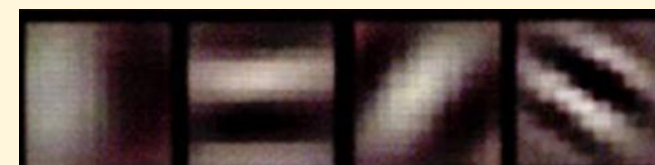
- 3차원 구조(이미지를 다루기 때문)
- **깊이(depth)**를 가짐
 - 합성곱을 거치며 특성맵이 만들어지고 이에 따라 중간 영상의 깊이가 달라짐

AlexNet 구조

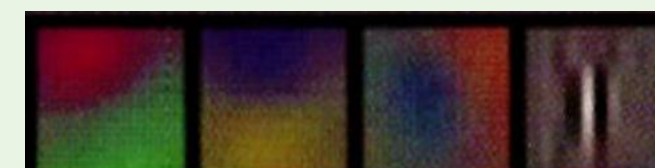
- ① GPU 두 개를 기반으로 한 병렬 구조
- ② 합성곱층 5개 + 완전연결층 3개



<GPU-1>: 주로 **컬러와 상관 없는 정보** 추출하기 위한 커널이 학습 됨



<GPU-2>: 주로 **컬러와 관련된 정보**를 추출하기 위한 커널이 학습 됨



AlexNet 개요

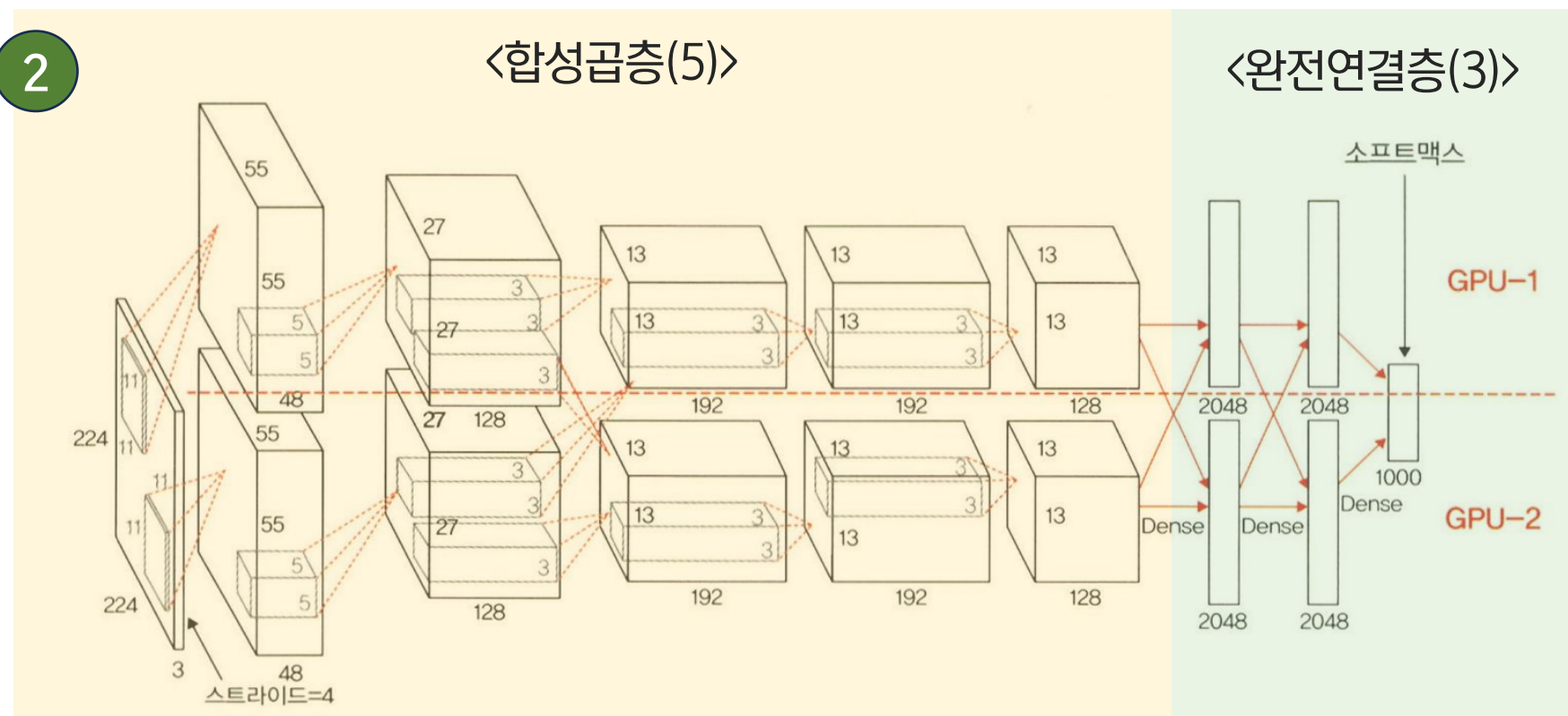
: LeNet-5의 CNN 구조를 크게 확장하여, 대규모 이미지 분류를 가능하게 한 대표적인 CNN 구조

CNN 복습

- 3차원 구조(이미지를 다루기 때문)
- **깊이(depth)**를 가짐
 - 합성곱을 거치며 특성맵이 만들어지고 이에 따라 중간 영상의 깊이가 달라짐

AlexNet 구조

- ① GPU 두 개를 기반으로 한 병렬 구조
- ② 합성곱층 5개 + 완전연결층 3개



⇒ LeNet-5보다 깊어지고 커진 구조
→ 파라미터 수가 많아 과적합 위험이 커지므로
Dropout 사용

AlexNet 세부 구조

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지(입력)	1	227 × 227	-	-	-
합성곱층	96	55 × 55	11 × 11	4	렐루(ReLU)
최대 풀링층	96	27 × 27	3 × 3	2	-
합성곱층	256	27 × 27	5 × 5	1	렐루(ReLU)
최대 풀링층	256	13 × 13	3 × 3	2	-
합성곱층	384	13 × 13	3 × 3	1	렐루(ReLU)
합성곱층	384	13 × 13	3 × 3	1	렐루(ReLU)
합성곱층	256	13 × 13	3 × 3	1	렐루(ReLU)
최대 풀링층	256	6 × 6	3 × 3	2	-
완전연결층	-	4096	-	-	렐루(ReLU)
완전연결층	-	4096	-	-	렐루(ReLU)
완전연결층	-	1000	-	-	소프트맥스(softmax)



ImageNet의 1000개 카테고리 분류를 위한 출력층
→ 마지막 FC에서 1000개의 출력값을 만들고,
소프트맥스를 통해 각 카테고리의 확률을 계산

입력: 227 × 227 × 3크기의 RGB 이미지
출력: 각 클래스에 해당하는 1000 × 1 확률 벡터를 출력

AlexNet 세부 구조

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지(입력)	1	227 × 227	-	-	-
합성곱층	96	55 × 55	11 × 11	4	렐루(ReLU)
최대 풀링층	96	27 × 27	3 × 3	2	-
합성곱층	256	27 × 27	5 × 5	1	렐루(ReLU)
최대 풀링층	256	13 × 13	3 × 3	2	-
합성곱층	384	13 × 13	3 × 3	1	렐루(ReLU)
합성곱층	384	13 × 13	3 × 3	1	렐루(ReLU)
합성곱층	256	13 × 13	3 × 3	1	렐루(ReLU)
최대 풀링층	256	6 × 6	3 × 3	2	-
완전연결층	-	4096	-	-	렐루(ReLU)
완전연결층	-	4096	-	-	렐루(ReLU)
완전연결층	-	1000	-	-	소프트맥스(softmax)

1

2

3

1

합성곱층을 여러 개 연속으로 쌓음
→ Pooling으로 크기를 줄이기 전에 **여러 번 특징을 가공**하여 더 복잡한 특징을 추출하기 위한 구조

2

특성 맵 수가 계속 증가하지 않음
→ 특성 맵 수가 깊이에 따라 **무조건 증가해야 하는 것은 아니며, 모델 설계에 따라 조절**할 수 있음

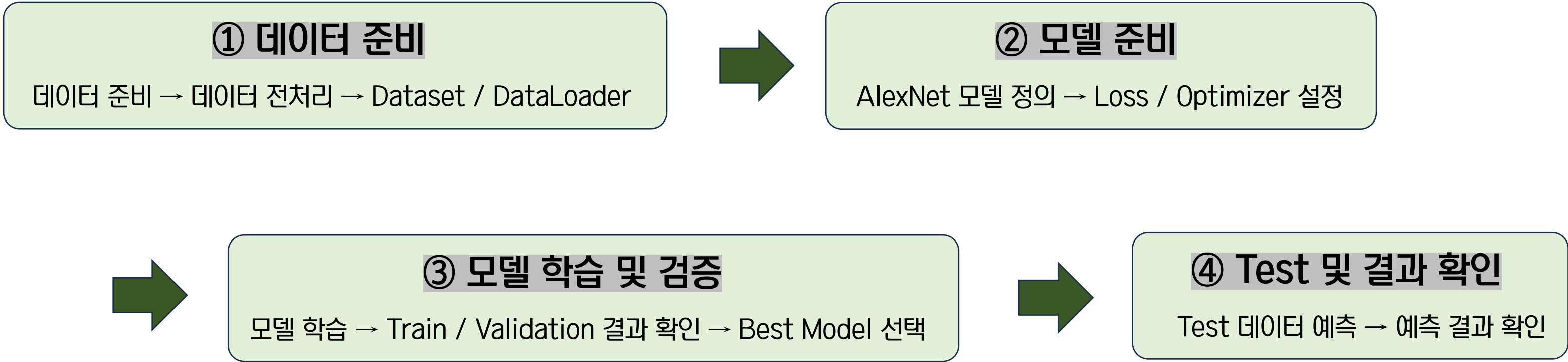
3

FC도 4096 → 4096으로 바로 줄어들지 않음
→ 4096개의 **뉴런을 두 층 유지**하며 **특징을 충분히 조합·가공**한 뒤, 마지막에 1000개 클래스로 분류

ImageNet의 1000개 카테고리 분류를 위한 출력층
→ 마지막 FC에서 1000개의 출력값을 만들고, 소프트맥스를 통해 각 카테고리의 확률을 계산

입력: 227 × 227 × 3크기의 RGB 이미지
출력: 각 클래스에 해당하는 1000 × 1 확률 벡터를 출력

AlexNet 실습 과정



이번 실습에서 변경한 부분

: AlexNet의 많은 파라미터를 고려하여 학습 데이터를 확대하고, 학습 과정을 확인할 수 있도록 일부 코드를 수정

항목	교재	이번 실습
Train 데이터	400장	5,000장
Validation 데이터	92장	980장
Test 데이터	10장	20장
Epoch	10	30
모델 기록	기본 학습	history 추가
모델 저장	마지막으로 학습한 모델	Best validation model 저장
Model Summary	torchsummary	torchinfo

데이터 준비

[코드 6-22] 필요한 라이브러리 호출

```
import torch
import torchvision
from torch.utils.data import DataLoader, Dataset
from torchvision import transforms
from torch import optim
import torch.nn as nn
import torch.nn.functional as F

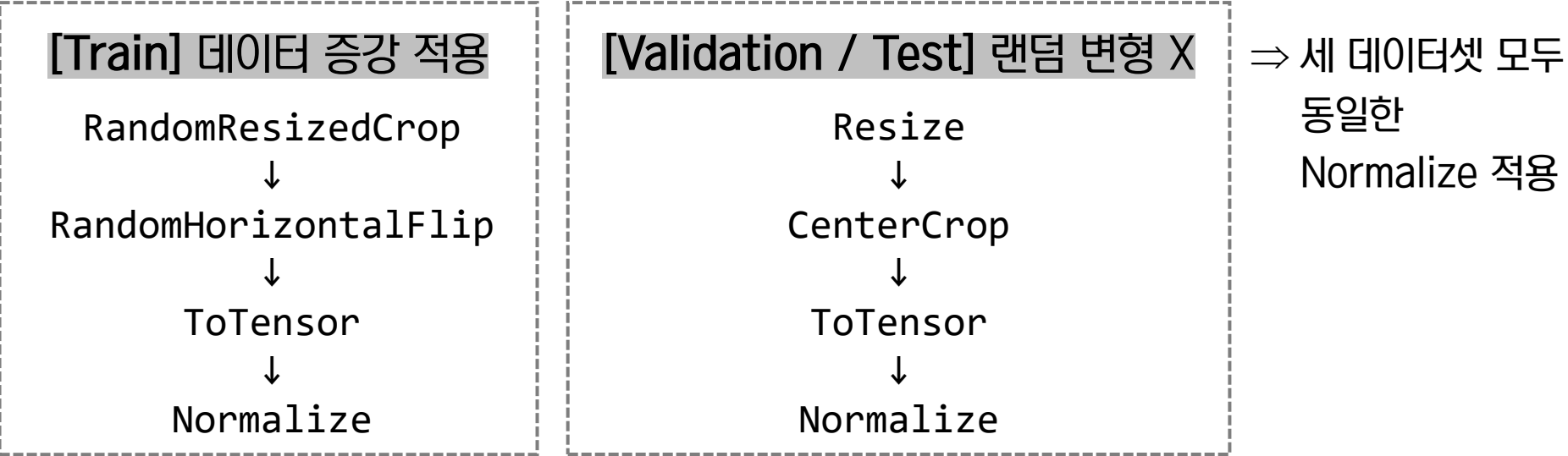
import os
import cv2
from PIL import Image
from tqdm.notebook import tqdm
import random
import time
import matplotlib.pyplot as plt

device = torch.device(
    "cuda:0" if torch.cuda.is_available() else "cpu"
)
```

[코드 6-23] 데이터 전처리

```
class ImageTransform():
    def __init__(self, resize, mean, std):
        self.data_transform = {
            'train': transforms.Compose([
                transforms.RandomResizedCrop(
                    resize, scale=(0.5, 1.0)
                ),
                transforms.RandomHorizontalFlip(),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ]),
            'val': transforms.Compose([
                transforms.Resize(256),
                transforms.CenterCrop(resize),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ])
        }

    def __call__(self, img, phase):
        return self.data_transform[phase](img)
```



데이터 준비

[추가 코드] 데이터 셋 준비

```
import shutil

shutil.unpack_archive(
    'dogs-vs-cats_ver2.zip',
    'dogs-vs-cats_ver2/'
)
```

Dogs vs Cats 데이터셋

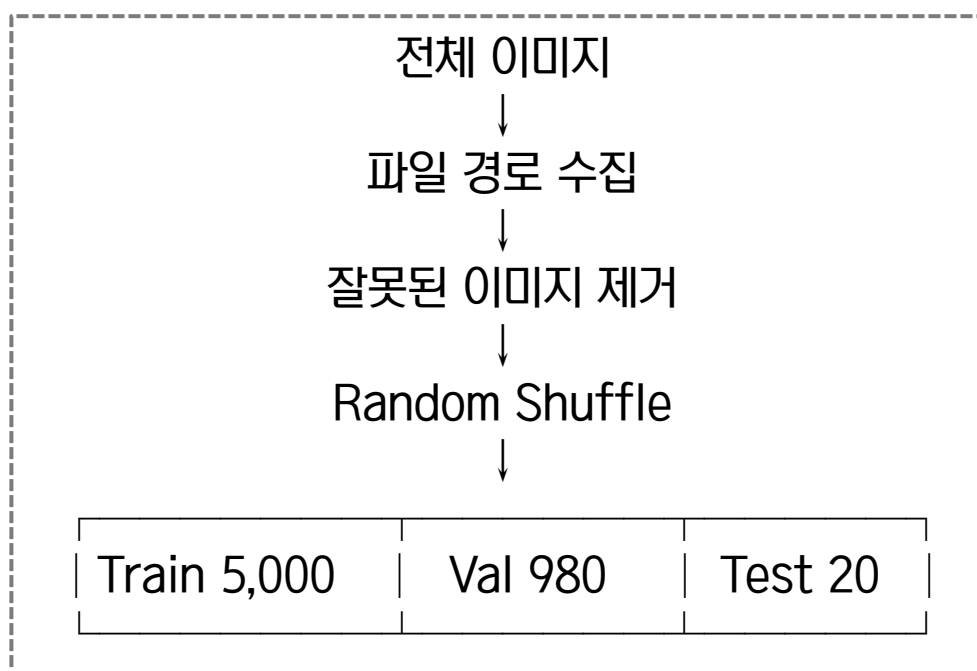
- Cat / Dog 이미지 사용
- Colab 환경에서 ZIP 파일 압축 해제

[코드 6-24] 데이터셋 분리

```
random.seed(42)
random.shuffle(correct_images_filepaths)

N_TEST = 20

train_images_filepaths = correct_images_filepaths[:5000]
val_images_filepaths = correct_images_filepaths[5000:-N_TEST]
test_images_filepaths = correct_images_filepaths[-N_TEST:]
```



데이터 준비

[코드 6-25] Custom Dataset 정의

```
class DogvsCatDataset(Dataset):
```

```
    def __init__(self, file_list,
                  transform=None,
                  phase='train'):
        self.file_list = file_list
        self.transform = transform
        self.phase = phase
```

```
    def __len__(self):
        return len(self.file_list)
```

```
    def __getitem__(self, idx):
```

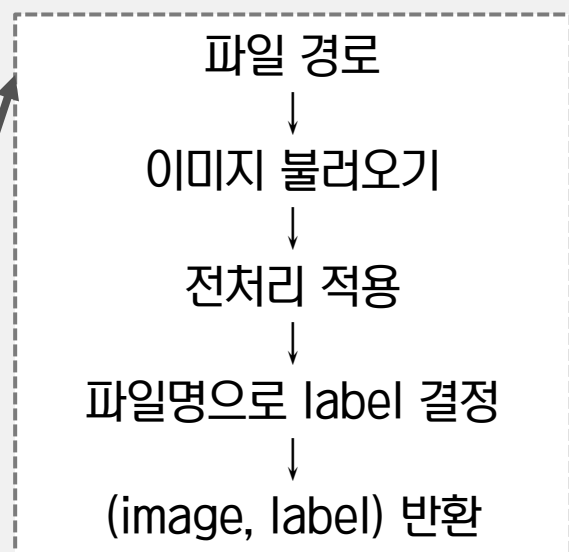
```
        img_path = self.file_list[idx]
        img = Image.open(img_path)
```

```
        img_transformed = self.transform(
            img, self.phase
        )
```

```
        label = img_path.split('/')[-1].split('.')[0]
```

```
        if label == 'dog':
            label = 1
        elif label == 'cat':
            label = 0
```

```
        return img_transformed, label
```



[코드 6-26] 입력 크기 및 학습 설정

```
size = 256
```

```
mean = (0.485, 0.456, 0.406)
```

```
std = (0.229, 0.224, 0.225)
```

```
batch_size = 32
```

Input

3 x 256 x 256

Batch

32 images

데이터 준비

[코드 6-27] 최종 데이터 생성 및 확인

```
train_dataset = DogvsCatDataset(
    train_images_filepaths,
    transform=ImageTransform(size, mean, std),
    phase='train'
)

val_dataset = DogvsCatDataset(
    val_images_filepaths,
    transform=ImageTransform(size, mean, std),
    phase='val'
)

test_dataset = DogvsCatDataset(
    test_images_filepaths,
    transform=ImageTransform(size, mean, std),
    phase='val'
)

index = 0
print(train_dataset.__getitem__(index)[0].size())
print(train_dataset.__getitem__(index)[1])
```

출력
torch.Size([3, 256, 256]) → RGB 이미지
1 → Dog

[코드 6-28] DataLoader로 Batch 구성

```
train_dataloader = DataLoader(
    train_dataset,
    batch_size=batch_size,
    shuffle=True
)

val_dataloader = DataLoader(
    val_dataset,
    batch_size=batch_size,
    shuffle=False
)

test_dataloader = DataLoader(
    test_dataset,
    batch_size=batch_size,
    shuffle=False
)

dataloader_dict = {'train': train_dataloader, 'val': val_dataloader}

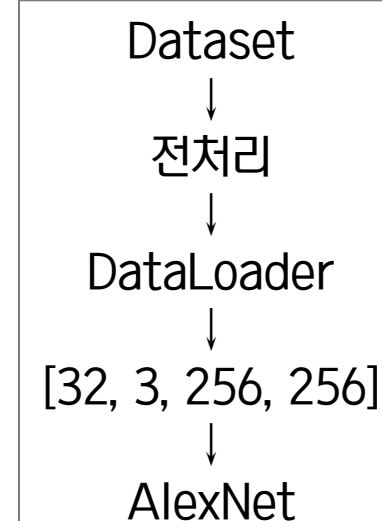
batch_iterator = iter(train_dataloader)
inputs, label = next(batch_iterator)

print(inputs.size())
print(label)
```

출력
torch.Size([32, 3, 256, 256]) → inputs
tensor([1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 1]) → 라벨 32개



데이터
준비완료!!!!!!
!!!!!!



모델 학습 준비

[코드 6-29] AlexNet 모델 정의

```
class AlexNet(nn.Module):
    def __init__(self) -> None:
        super(AlexNet, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=11, stride=4, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(64, 192, kernel_size=5, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(192, 384, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(384, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(256, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
        )
        self.avgpool = nn.AdaptiveAvgPool2d((6, 6))
        self.classifier = nn.Sequential(
            nn.Dropout(),
            nn.Linear(256 * 6 * 6, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(),
            nn.Linear(4096, 512),
            nn.ReLU(inplace=True),
            nn.Linear(512, 2),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.features(x)
        x = self.avgpool(x)
        x = torch.flatten(x, 1)
        x = self.classifier(x)
        return x
```

| 전체 흐름

코드	AlexNet에서의 역할
<code>self.features</code>	합성곱층 + ReLU + Pooling → 특징 추출
<code>self.avgpool</code>	특징 맵의 크기를 6 × 6으로 조정
<code>torch.flatten()</code>	특징 맵을 1차원으로 변환
<code>self.classifier</code>	완전연결층 → 최종 분류

(* 이번 실습에서는 pretrained weight를 사용하지 않고, AlexNet 구조를 직접 정의하여 처음부터 학습합니다.)

모델 학습 준비

[코드 6-29] AlexNet 모델 정의

```
class AlexNet(nn.Module):
    def __init__(self) -> None:
        super(AlexNet, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=11, stride=4, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(64, 192, kernel_size=5, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(192, 384, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(384, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(256, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
        )
        self.avgpool = nn.AdaptiveAvgPool2d((6, 6))
        self.classifier = nn.Sequential(
            nn.Dropout(),
            nn.Linear(256 * 6 * 6, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(),
            nn.Linear(4096, 512),
            nn.ReLU(inplace=True),
            nn.Linear(512, 2),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.features(x)
        x = self.avgpool(x)
        x = torch.flatten(x, 1)
        x = self.classifier(x)
        return x
```

💡 Adaptive Average Pooling

일반 AvgPool2d

- 커널 크기 / stride 등 **과정을 직접 지정**
- 결과 크기가 결정됨

AdaptiveAvgPool2d

- 원하는 출력 크기 **결과를 지정**
- 입력 크기에 맞춰 pooling 수행
- ⇒ 입력 크기가 달라져도 원하는 출력 크기로 맞출 수 있어, CNN 뒤에 완전연결층을 사용하는 경우 유용
- ⇒ 현재 코드에서는 특징 맵의 크기를 최종적으로 6 × 6으로 맞추기 위해 사용

📌 Flatten된 9216개의 특징을 완전연결층에서 4096 → 512로 줄여가면서 최종적으로 2개의 클래스에 대한 출력값을 생성

모델 학습 준비

[코드 6-30] 모델 객체 생성

```
model = AlexNet()  
model.to(device)
```

[코드 6-31] Loss와 Optimizer 설정

```
optimizer = optim.SGD(  
    model.parameters(),  
    lr=0.001,  
    momentum=0.9  
)  
  
criterion = nn.CrossEntropyLoss()
```

Optimizer

: SGD + Momentum

Loss

: Cross Entropy

모델 학습 준비

[코드 6-32] torchinfo로 모델 확인

```
!pip install torchinfo

from torchinfo import summary

summary(
    model,
    input_size=(1, 3, 256, 256)
)
```

 **torchinfo**

torchsummary → torchinfo

- layer별 output shape / parameter 확인

- torchinfo는 batch 차원 포함

➡ # 출력

Layer (type:depth-idx)	Output Shape	Param #
AlexNet	[1, 2]	--
└Sequential: 1-1	[1, 256, 7, 7]	--
└Conv2d: 2-1	[1, 64, 63, 63]	23,296
└ReLU: 2-2	[1, 64, 63, 63]	--
└MaxPool2d: 2-3	[1, 64, 31, 31]	--
└Conv2d: 2-4	[1, 192, 31, 31]	307,392
└ReLU: 2-5	[1, 192, 31, 31]	--
└MaxPool2d: 2-6	[1, 192, 15, 15]	--
└Conv2d: 2-7	[1, 384, 15, 15]	663,936
└ReLU: 2-8	[1, 384, 15, 15]	--
└Conv2d: 2-9	[1, 256, 15, 15]	884,992
└ReLU: 2-10	[1, 256, 15, 15]	--
└Conv2d: 2-11	[1, 256, 15, 15]	590,080
└ReLU: 2-12	[1, 256, 15, 15]	--
└MaxPool2d: 2-13	[1, 256, 7, 7]	--
└AdaptiveAvgPool2d: 1-2	[1, 256, 6, 6]	--
└Sequential: 1-3	[1, 2]	--
└Dropout: 2-14	[1, 9216]	--
└Linear: 2-15	[1, 4096]	37,752,832
└ReLU: 2-16	[1, 4096]	--
└Dropout: 2-17	[1, 4096]	--
└Linear: 2-18	[1, 512]	2,097,664
└ReLU: 2-19	[1, 512]	--
└Linear: 2-20	[1, 2]	1,026

Total params: 42,321,218
Trainable params: 42,321,218
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 908.99

Input size (MB): 0.79
Forward/backward pass size (MB): 5.16
Params size (MB): 169.28
Estimated Total Size (MB): 175.23

모델 학습 준비 완료!!!

Data

- ✓ 5,000 Train
- ✓ 980 Validation
- ✓ 20 Test

Model

- ✓ AlexNet

Loss

- ✓ CrossEntropyLoss

Optimizer

- ✓ SGD + Momentum

모델 학습

[코드 6-33] 모델 학습 함수 정의: 기본 흐름

```
def train_model(model, dataloader_dict, criterion, optimizer, num_epoch):
    since = time.time()

    history = {'train_loss': [], 'train_acc': [], 'val_loss': [], 'val_acc': []}
    best_acc = 0.0

    for epoch in range(num_epoch):
        print('Epoch {}/{}'.format(epoch + 1, num_epoch))
        print('-'*20)

        for phase in ['train', 'val']:
            if phase == 'train':
                model.train()
            else:
                model.eval()

            epoch_loss = 0.0
            epoch_corrects = 0

            for inputs, labels in tqdm(dataloader_dict[phase]):
                inputs = inputs.to(device)
                labels = labels.to(device)
                optimizer.zero_grad()

                with torch.set_grad_enabled(phase == 'train'):
                    outputs = model(inputs)
                    _, preds = torch.max(outputs, 1)
                    loss = criterion(outputs, labels)

                    if phase == 'train':
                        loss.backward()
                        optimizer.step()

                epoch_loss += loss.item() * inputs.size(0)
                epoch_corrects += torch.sum(preds == labels)
```

2

⇒ AlexNet에는 Dropout이 있기 때문에 중요

```
epoch_loss = epoch_loss / len(dataloader_dict[phase].dataset)
epoch_acc = epoch_corrects.double() / len(dataloader_dict[phase].dataset)

history[phase + '_loss'].append(epoch_loss)
history[phase + '_acc'].append(epoch_acc)
print('{} Loss: {:.4f} Acc: {:.4f}'.format(phase, epoch_loss, epoch_acc))

if phase == 'val' and epoch_acc > best_acc:
    best_acc = epoch_acc
    torch.save(model.state_dict(), '/content/drive/MyDrive/alexnet_best.pth')

time_elapsed = time.time() - since
print('Training complete in {:.0f}m {:.0f}s'.format(
    time_elapsed // 60, time_elapsed % 60))
print('Best val Acc: {:.4f}'.format(best_acc))

return model, history
```

1

[전체 흐름]

Epoch
↓
Train
↓
Validation
↓
Loss / Accuracy 기록
↓
Best Model 저장

모델 학습

[코드 6-33] 모델 학습 함수 정의(추가)

```
def train_model(model, dataloader_dict, criterion, optimizer, num_epoch):  
    since = time.time()
```

```
    history = {'train_loss': [], 'train_acc': [], 'val_loss': [], 'val_acc': []}  
    best_acc = 0.0
```

⇒ 기록 저장할 변수

```
    for epoch in range(num_epoch):  
        print('Epoch {}/{}'.format(epoch + 1, num_epoch))  
        print('-'*20)
```

```
        for phase in ['train', 'val']:  
            if phase == 'train':  
                model.train()  
            else:  
                model.eval()
```

```
        epoch_loss = 0.0  
        epoch_corrects = 0
```

```
        for inputs, labels in tqdm(dataloader_dict[phase]):  
            inputs = inputs.to(device)  
            labels = labels.to(device)  
            optimizer.zero_grad()
```

```
            with torch.set_grad_enabled(phase == 'train'):  
                outputs = model(inputs)  
                _, preds = torch.max(outputs, 1)  
                loss = criterion(outputs, labels)
```

```
            if phase == 'train':  
                loss.backward()  
                optimizer.step()
```

```
            epoch_loss += loss.item() * inputs.size(0)  
            epoch_corrects += torch.sum(preds == labels)
```

```
        epoch_loss = epoch_loss / len(dataloader_dict[phase].dataset)  
        epoch_acc = epoch_corrects.double() / len(dataloader_dict[phase].dataset)
```

```
        history[phase + '_loss'].append(epoch_loss)  
        history[phase + '_acc'].append(epoch_acc)  
        print('{} Loss: {:.4f} Acc: {:.4f}'.format(phase, epoch_loss, epoch_acc))
```

⇒ Loss와 acc 기록 저장

```
        if phase == 'val' and epoch_acc > best_acc:  
            best_acc = epoch_acc  
            torch.save(model.state_dict(), '/content/drive/MyDrive/alexnet_best.pth')
```

```
    time_elapsed = time.time() - since  
    print('Training complete in {:.0f}m {:.0f}s'.format(  
        time_elapsed // 60, time_elapsed % 60))  
    print('Best val Acc: {:.4f}'.format(best_acc))
```

⇒ val 정확도가 최고일 때 가중치 저장

```
    return model, history
```

모델 학습

[코드 6-34] AlexNet 학습

```
num_epoch = 30

model, history = train_model(
    model,
    dataloader_dict,
    criterion,
    optimizer,
    num_epoch
)
```

```
Epoch 1/30
-----
100% ██████████ 157/157 [00:36<00:00, 5.83it/s]
train Loss: 0.6933 Acc: 0.4940
100% ██████████ 31/31 [00:05<00:00, 7.14it/s]
val Loss: 0.6929 Acc: 0.4969
Epoch 2/30
-----
100% ██████████ 157/157 [00:35<00:00, 4.88it/s]
train Loss: 0.6930 Acc: 0.5102
100% ██████████ 31/31 [00:05<00:00, 5.27it/s]
val Loss: 0.6929 Acc: 0.4888
...
(생략)
...
Epoch 30/30
-----
100% ██████████ 157/157 [00:32<00:00, 5.07it/s]
train Loss: 0.5385 Acc: 0.7372
100% ██████████ 31/31 [00:05<00:00, 5.36it/s]
val Loss: 0.5654 Acc: 0.7031
Training complete in 20m 1s
Best val Acc: 0.7735
```

[추가 코드] 학습 결과 시각화

```
history['train_acc'] = [a.item() if torch.is_tensor(a) else a for
a in history['train_acc']]
history['val_acc'] = [a.item() if torch.is_tensor(a) else a for a
in history['val_acc']]

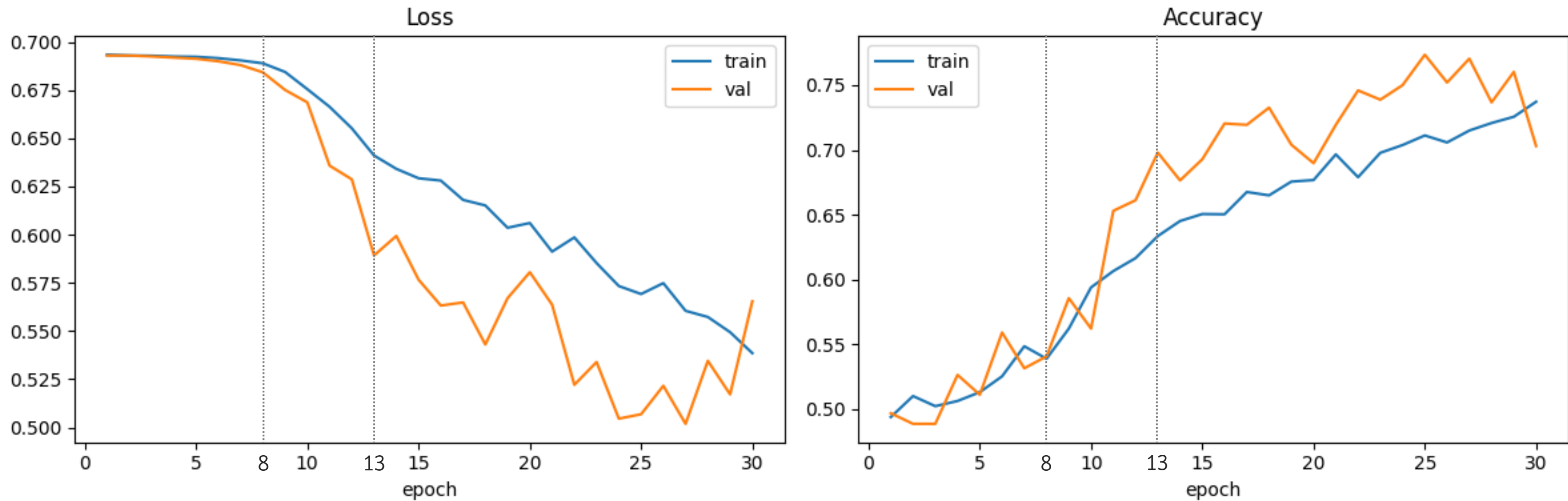
epochs = range(1, len(history['train_loss']) + 1)
fig, ax = plt.subplots(1, 2, figsize=(12, 4))

ax[0].plot(epochs, history['train_loss'], label='train')
ax[0].plot(epochs, history['val_loss'], label='val')
ax[0].set_title('Loss')
ax[0].set_xlabel('epoch')
ax[0].legend()

ax[1].plot(epochs, history['train_acc'], label='train')
ax[1].plot(epochs, history['val_acc'], label='val')
ax[1].set_title('Accuracy')
ax[1].set_xlabel('epoch')
ax[1].legend()

plt.tight_layout()
plt.show()
```

모델 학습 결과



Best Val Acc:

77.35%

(Epoch 25)

1 : 1 ~ 8 epoch

- Loss ≈ 0.693 , Accuracy $\approx 50\%$ (무작위 수준)
- 학습이 거의 진행되지 않음

2 : 9 ~ 13 epoch

- Train Loss $0.684 \rightarrow 0.641$
- Val Accuracy $59\% \rightarrow 70\%$
- 가장 가파르게 개선된 구간

3 : 14 ~ 30 epoch

- Train Loss $0.634 \rightarrow 0.539$ (지속 감소)
- Train Accuracy $64\% \rightarrow 74\%$ (지속 증가)
- Validation은 등락이 있지만 전체적으로 개선

Test

[추가 코드] Best Model을 최종 모델로 사용

```
model.load_state_dict(torch.load('/content/drive/MyDrive/alexnet_best.pth'))
model.to(device)
```

[코드 6-35] Test 데이터 예측 및 저장

```
with torch.no_grad():
    ...
    img = transform(img, phase='val')
    img = img.unsqueeze(0)
    img = img.to(device)

    model.eval()
    outputs = model(img)
    preds = F.softmax(outputs, dim=1)[: , 1]

res = pd.DataFrame({
    'id': id_list,
    'label': pred_list
})

res.to_csv('/content/alexnet.csv', index=False)
```

⇒ dog에 해당하는 확률

[코드 6-36] Test 데이터 예측 결과 확인

```
res.head(20)
```

	id	label
0	2736	0.295641
1	2022	0.558485
2	2712	0.280349
3	261	0.238680
4	1689	0.619210
5	1217	0.239947
6	1231	0.170391
7	1408	0.607132
8	2651	0.619390
9	1639	0.279412
10	2318	0.711287
11	588	0.624312
12	1753	0.256305
13	2026	0.546611
14	2643	0.660802
15	2803	0.404535
16	326	0.293463
17	1181	0.529009
18	1819	0.422407
19	312	0.226089

⇒ Label: dog에 해당하는 확률

Test

[코드 6-37] 예측 결과 시각화 함수 정의

```
class_ = {0: 'cat', 1: 'dog'}

def display_image_grid(images_filepaths, cols=5):
    ...

    label = res['label'].iloc[i] (* 수정)

    if label > 0.5:
        label = 1
    else:
        label = 0

    true = 1 if os.path.basename(image_filepath).startswith('dog') else 0
    color = 'green' if label == true else 'red'

    ax.ravel()[i].imshow(image)
    ax.ravel()[i].set_title(
        f'pred: {class_[label]}\ntrue: {class_[true]}',
        color=color
    )
    ...
```

⇒ dog 확률을 기준으로 최종 클래스를 결정

- 예측 확률 0.5를 기준으로 cat과 dog로 변환
- 파일명을 통해 실제 정답 확인
- 정답이면 초록, 오답이면 빨강

Test 결과

[코드 6-38] 예측 결과 출력

```
display_image_grid(test_images_filepaths)
```



14 / 20
정답 개수 70%

- 교재 결과**
- val acc $\approx$ 0.51
 - 테스트 데이터의 모든 이미지를 cat으로 예측
- 이번 실습**
- best val acc = 0.77
 - 테스트 데이터 14 / 20 정답
 - 교재 결과보다 개선된 분류 결과 확인

- 더 나은 성능을 위해**
- 데이터 증강
 - 학습 데이터 증가
 - Epoch 증가

6.1.3 VGGNet

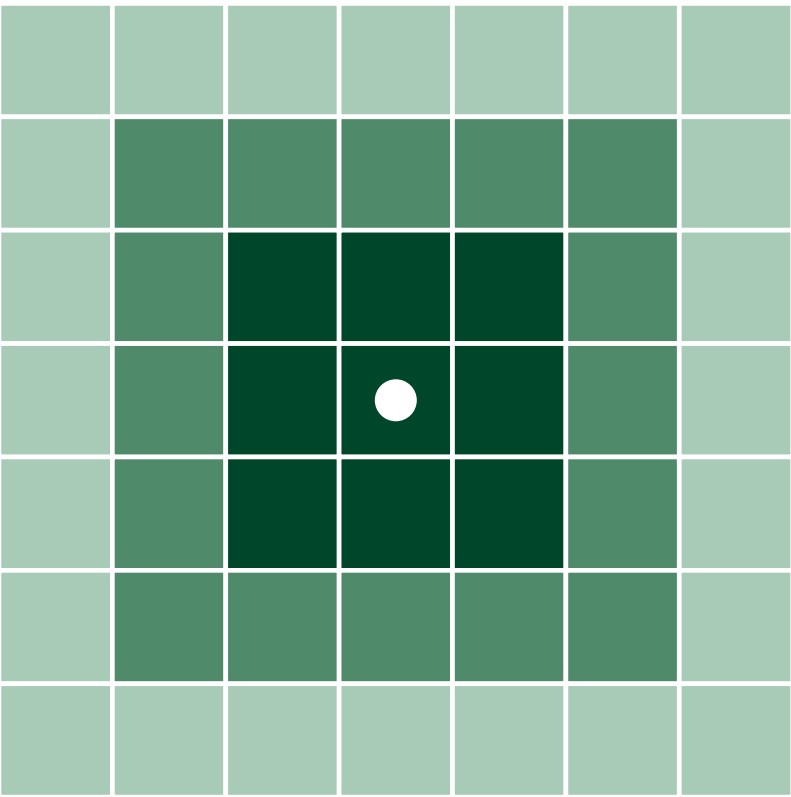


6.1.3 VGGNet 탄생 배경

- 옥스퍼드대 VGG 연구팀 (시모니안, 지서만)
- “Very Deep Convolutional Networks for Large-Scale Image Recognition” (ICLR 2015)
- ILSVRC 2014 분류 부문 2위, 위치 추정 부문 1위
- 깊이의 영향만 보기 위해 나머지 조건은 최대한 고정
- 합성곱 필터를 가장 작은 3×3 으로 통일해서 파라미터 수와 훈련 시간을 줄임
- 같은 블록을 반복하는 단순한 구조 (VGG11 ~ VGG19)



6.1.3 3 × 3 필터



- 3 × 3 합성곱 1번 → 3 × 3
- 2번 쌓으면 → 5 × 5
- 3번 쌓으면 → 7 × 7

출력 1픽셀이 보는 입력 범위 (수용 영역)

	7 × 7 합성곱 1개	3 × 3 합성곱 3개
수용 영역	7 × 7	7 × 7 (같음)
파라미터 (C → C)	49C ²	3 × 9C ² = 27C ²
ReLU (비선형성)	1번	3번

- 작은 필터를 깊게 쌓으면 큰 필터 하나와 같은 범위를 보면서도 파라미터가 적고 표현력은 커짐
 - 5 × 5 하나 대신 3 × 3 두 번을 써도 25C² → 18C²로 약 28% 감소
 - 같은 범위를 보면서 파라미터는 약 45% 줄고, ReLU는 3번 들어감

6.1.3 VGG16 구조

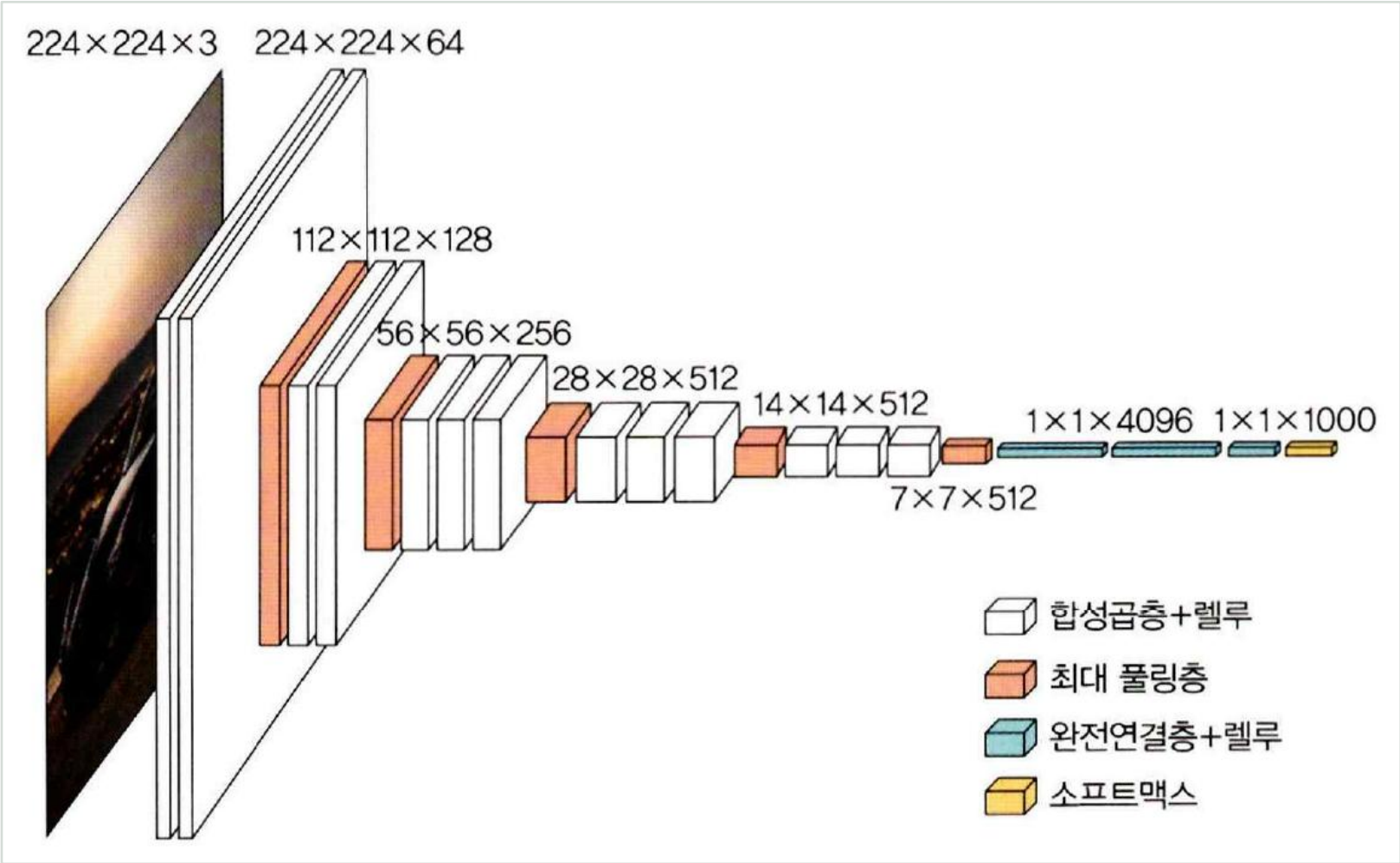


그림 6-13 VGG16 구조

블록	구성	출력 크기
Block 1	conv3-64 × 2 → pool	112 × 112 × 64
Block 2	conv3-128 × 2 → pool	56 × 56 × 128
Block 3	conv3-256 × 3 → pool	28 × 28 × 256
Block 4	conv3-512 × 3 → pool	14 × 14 × 512
Block 5	conv3-512 × 3 → pool	7 × 7 × 512
FC	4096 → 4096 → 1000	softmax

- 합성곱 : 3 × 3, stride 1, padding 1 (크기 유지)
- 최대 풀링 : 2 × 2, stride 2 (크기 ½)
- 채널 수는 블록마다 2배 (64 → 512)
- 합성곱 13 + 완전연결 3 = **16층**
- 파라미터 약 **1억 3,836만 개**

6.1.3 VGG11/13/16/19

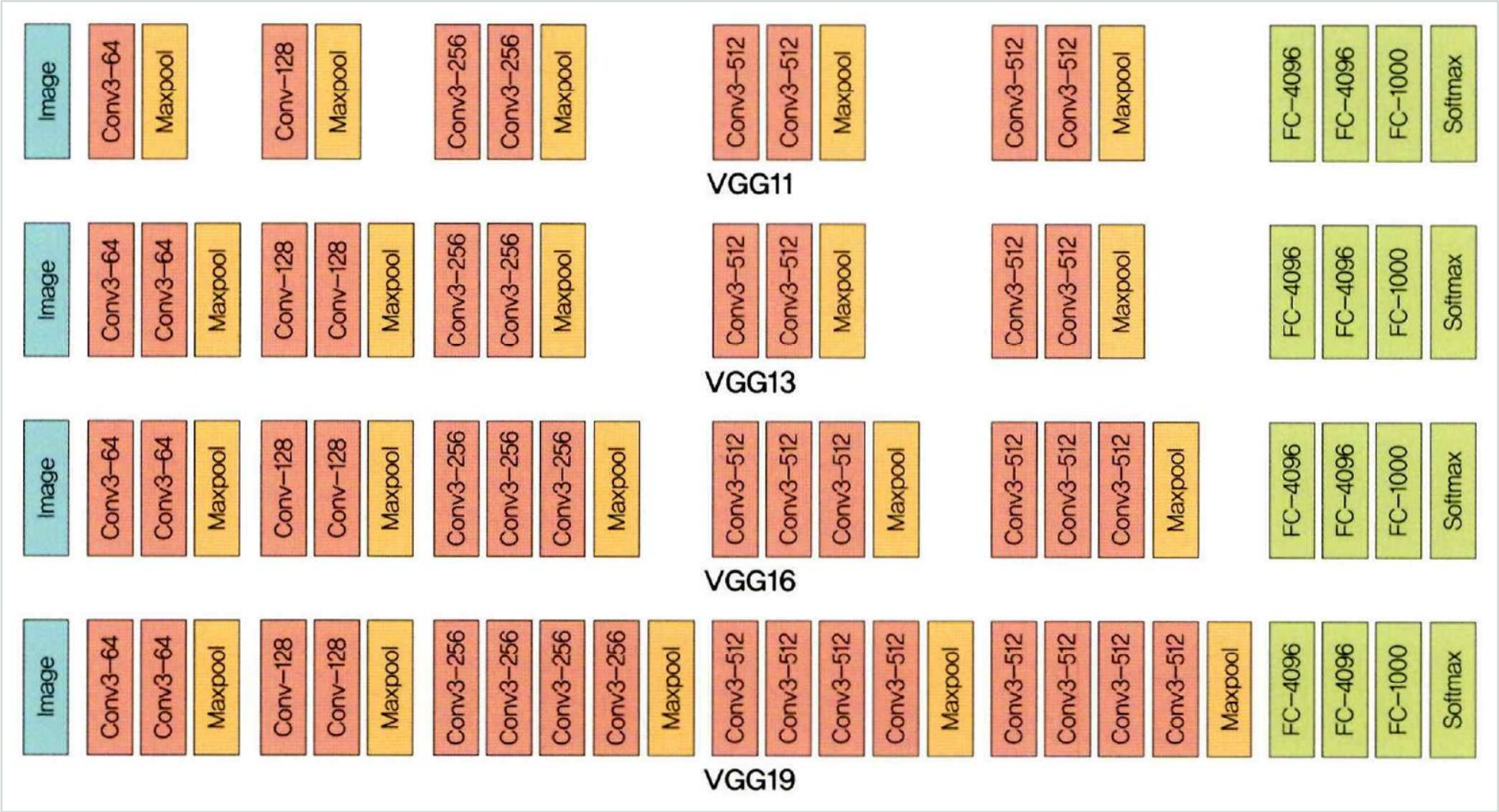


그림 6-15 VGG의 다양한 모델에 대한 네트워크

VGG-N의 N = 가중치가 있는 층 수

모델	합성곱	FC	파라미터
VGG11	8	3	1.33억
VGG13	10	3	1.33억
VGG16	13	3	1.38억
VGG19	16	3	1.44억

- 'M'(최대 풀링)은 모두 5개이고 층 수에는 포함 X
 - 교재 주석의 '+3(풀링층)' 은 FC층 3개를 뜻함
- 층이 늘어도 파라미터 차이가 작은 건 대부분이 FC층에 있기 때문

```
vgg11_config = [64, 'M', 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M']
vgg13_config = [64, 64, 'M', 128, 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M']
vgg16_config = [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 'M', 512, 512, 512, 'M', 512, 512, 512, 'M']
vgg19_config = [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 256, 'M', 512, 512, 512, 512, 'M', 512, 512, 512, 512, 'M']
```

6.1.3 라이브러리 호출과 객체 복사

코드 6-39 라이브러리 호출

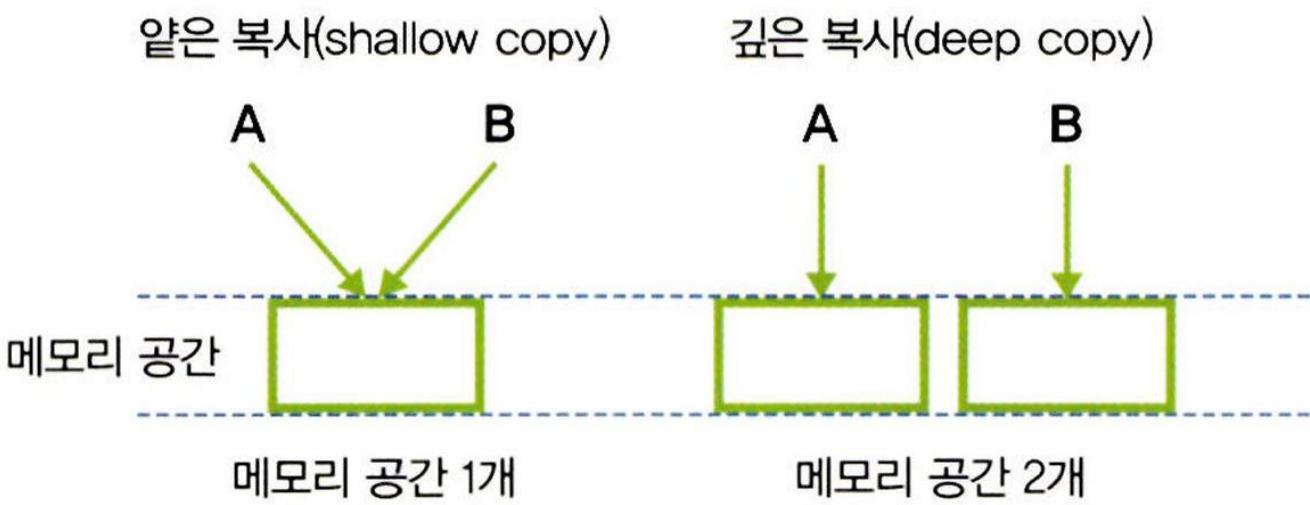
```
import copy
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torch.utils.data as data
import torchvision
import torchvision.transforms as transforms
import torchvision.datasets as Datasets

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

대입 vs 얇은 복사 vs 깊은 복사

original = [[1, 2], 3] 을 복사한 copy_o를 바꾸면?

	대입 =	copy.copy()	copy.deepcopy()
새 객체	같은 객체	바깥만 새로	안쪽까지 새로
copy_o[0] = 100	원본도 바뀜	원본 유지	원본 유지
copy_o[0].append(4)	원본도 바뀜	원본도 바뀜	원본 유지



뒤에서 검증 데이터를 훈련 데이터와 완전히 분리할 때 copy.deepcopy()를 사용 (코드 6-50)

6.1.3 VGG 모델 정의

#VGG 모델 정의

```
class VGG(nn.Module):
    def __init__(self, features, output_dim):
        super().__init__()
        self.features = features
        self.avgpool = nn.AdaptiveAvgPool2d(7)
        self.classifier = nn.Sequential(
            nn.Linear(512*7*7, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(p=0.5),
            nn.Linear(4096, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(p=0.5),
            nn.Linear(4096, output_dim)
        )
    def forward(self, x):
        x = self.features(x)
        x = self.avgpool(x)
        h = x.view(x.shape[0], -1)
        x = self.classifier(h)
        return x, h
```

1. features : 특징 추출부

get_vgg_layers()로 만든 합성곱 / 풀링 묶음을 받아서 사용
config만 바꾸면 VGG11~19를 같은 클래스로 만들 수 있음

2. avgpool : AdaptiveAvgPool2d(7)

입력 크기와 상관없이 출력을 7 × 7로 맞춤
256 × 256 입력의 8 × 8도 7 × 7로 바뀜 ($512 \times 7 \times 7 = 25,088$)

3. classifier : 완전연결층

$25,088 \rightarrow 4,096 \rightarrow 4,096 \rightarrow \text{output_dim}$ (= 2, 개 / 고양이)

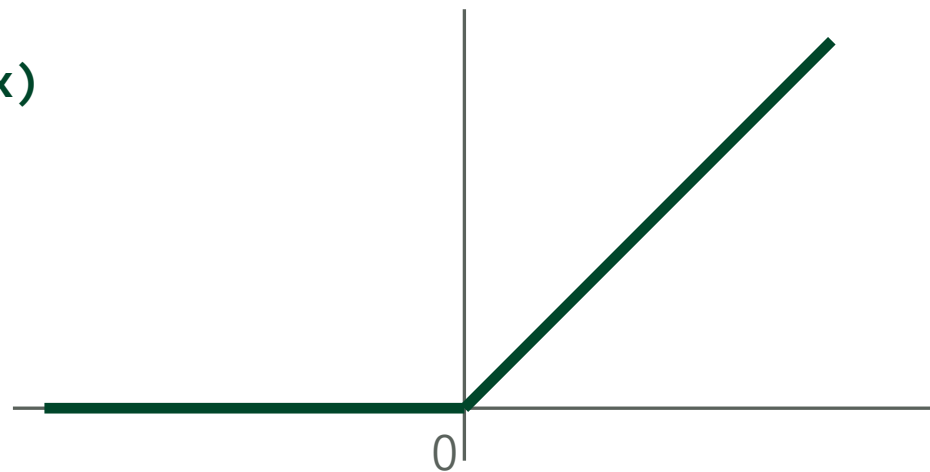
4. forward : 값 2개 반환

출력 x와 펼친 특징 h를 같이 반환해서 학습 코드에서는
`y_pred, _ = model(x)`

6.1.3 ReLU와 드롭아웃을 쓰는 이유

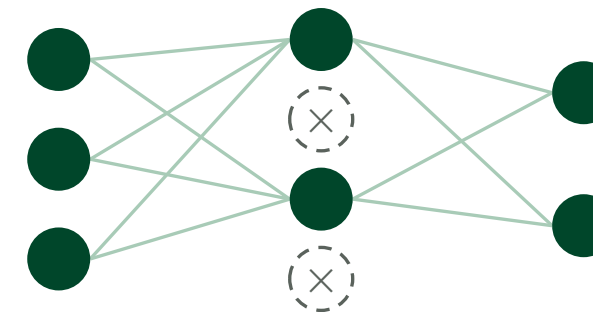
ReLU(inplace=True)

$$y = \max(0, x)$$

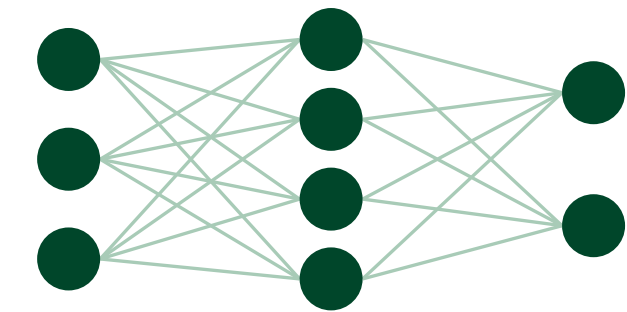


- 비선형성 추가
- 양수 구간 기울기가 1이라 기울기 소실이 덜하고 계산도 단순
- inplace=True : 입력 텐서를 그 자리에서 덮어서 메모리 절약

Dropout(0.5)



학습 시 (model.train())



평가 시 (model.eval())

- 학습 때마다 뉴런의 50%를 무작위로 끄
특정 뉴런에 기대지 않게 해서 과적합을 막음
- 평가 때는 모든 뉴런 사용 (model.eval())이 자동으로 꺼 줌)
- 파라미터가 FC층에 몰려 있기 때문에 FC층에만

우리 VGG11 (출력 2) 파라미터 128,780,034개

완전연결층(FC) 119,554,050개 (92.8%)

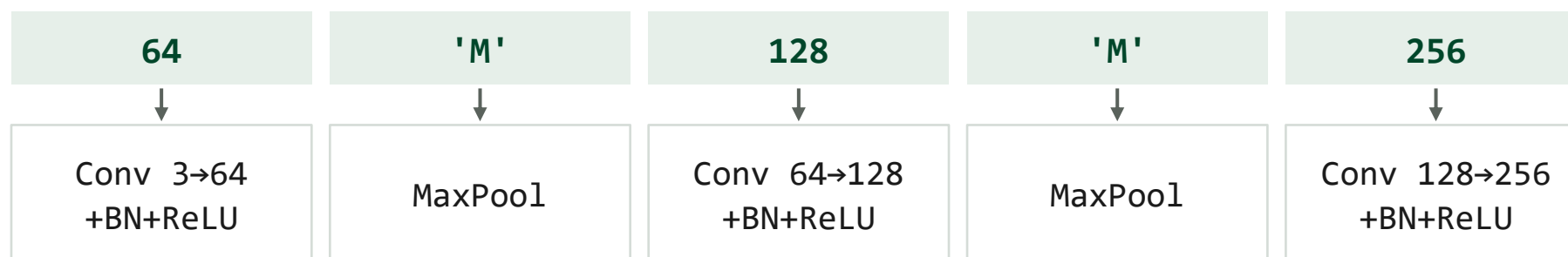
▲ 합성곱 + 배치 정규화 9,225,984개 (7.2%)

6.1.3 VGG 계층 정의 (get_vgg_layers)

#VGG 계층 정의

```
def get_vgg_layers(config, batch_norm):
    layers = []
    in_channels = 3
    for c in config:
        assert c == 'M' or isinstance(c, int)
        if c == 'M':
            layers += [nn.MaxPool2d(kernel_size=2)]
        else:
            conv2d = nn.Conv2d(in_channels, c, kernel_size=3, padding=1)
            if batch_norm:
                layers += [conv2d, nn.BatchNorm2d(c), nn.ReLU(inplace=True)]
            else:
                layers += [conv2d, nn.ReLU(inplace=True)]
            in_channels = c
    return nn.Sequential(*layers)
```

vgg11_config 앞부분이 계층으로 바뀌는 과정



동작 순서

1. in_channels = 3 (RGB)에서 시작
2. 'M' → MaxPool2d(2) : 크기 절반
3. 숫자 c → Conv2d(in_channels → c, 3 × 3, padding=1) + [BatchNorm2d] + ReLU
4. in_channels = c : 이번 출력 채널이 다음 층의 입력 채널
5. nn.Sequential(*layers) : 리스트를 풀어서 하나의 모듈로

assert 문

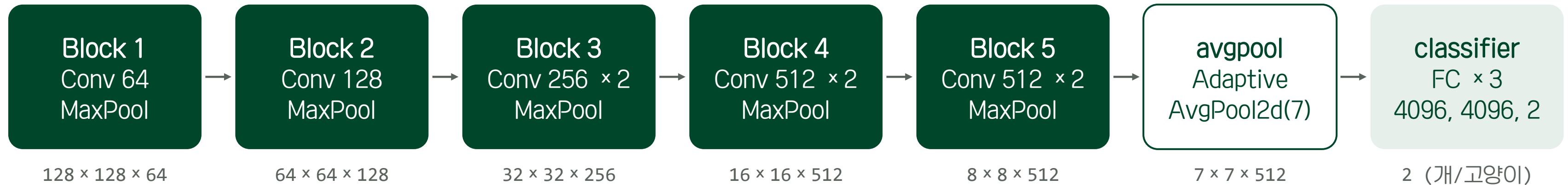
c가 'M'도 아니고 정수도 아니면 **AssertionError**
config에 잘못된 값이 들어오면 바로 멈춤

```
a = 1
assert a == 11          # → AssertionError
isinstance(1, int)      # True
isinstance(1.2, int)    # False
```

6.1.3 VGG11 모델 생성

```
vgg11_layers = get_vgg_layers(vgg11_config, batch_norm=True)
OUTPUT_DIM = 2 # 개와 고양이, 두 개의 클래스
model = VGG(vgg11_layers, OUTPUT_DIM)
```

VGG11 + 배치 정규화, 입력 $256 \times 256 \times 3$ 일 때 출력 크기



Conv = Conv2d(3×3 , padding=1) → BatchNorm2d → ReLU / MaxPool = 2×2 , stride 2 (크기 $\frac{1}{2}$)

print(model) 결과

features (0)~(28)
→ Conv-BN-ReLU × 8, MaxPool × 5
→ avgpool → classifier (0)~(6)

배치 정규화 (BatchNorm2d)

입력 분포를 평균 0, 표준편차 1로 맞춤
학습이 안정적이고 빨라짐
(8장에서 자세히)

총 파라미터 128,780,034개

그중 FC층이 92.8%
사전 훈련 모델 불러오기 :
`models.vgg11_bn(pretrained=True)`

6.1.3 데이터 준비 ① 이미지 전처리

#이미지 데이터 전처리

```
train_transforms = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.RandomRotation(5),
    transforms.RandomHorizontalFlip(0.5),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

test_transforms = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])
```

증강(augmentation)은 **훈련 데이터에만** 적용
검증, 테스트는 매번 같은 입력으로 평가해야 공정함

Resize((256, 256))

모든 이미지를 256 × 256 크기로 통일



RandomRotation(5)

-5° ~ +5° 사이로 무작위 회전

훈련만



RandomHorizontalFlip(0.5)

50% 확률로 좌우 반전

훈련만



ToTensor()

이미지(H × W × C, 0~255) → 텐서(C × H × W, 0~1)



Normalize(mean, std)

채널별 $(x - \text{mean}) / \text{std}$

Normalize 값은 ImageNet 데이터셋의 채널별 평균, 표준편차 (사전 훈련 모델과 같은 기준)

6.1.3 데이터 준비 ② 데이터셋 불러오기와 분할

ImageFolder : 폴더 구조 그대로 불러오기

```
catanddog/
├── train/
│   ├── Cat/      # 폴더 이름 = 클래스 (0)
│   └── Dog/      # 클래스 (1)
└── test/
    ├── Cat/
    └── Dog/
```

#ImageFolder를 이용해서 데이터셋 불러오기

```
train_path = '/content/drive/MyDrive/euron/4주차/080289-main/chap06/data/catanddog/train'
test_path = '/content/drive/MyDrive/euron/4주차/080289-main/chap06/data/catanddog/test'
```

```
train_dataset = torchvision.datasets.ImageFolder(
    train_path,
    transform=train_transforms
)
```

```
test_dataset = torchvision.datasets.ImageFolder(
    test_path,
    transform=test_transforms
)
```

```
print(len(train_dataset)), print(len(test_dataset))
```

- classes = ['Cat', 'Dog'], 폴더 이름 알파벳순으로 라벨 0, 1이 자동으로 붙음

훈련 / 검증 분할 (9 : 1)

훈련 폴더 이미지 529장

훈련 476장 (90%)	53
+ 테스트 12장 (test 폴더)	검증 53장 (10%) ▲

#훈련과 검증 데이터 분할

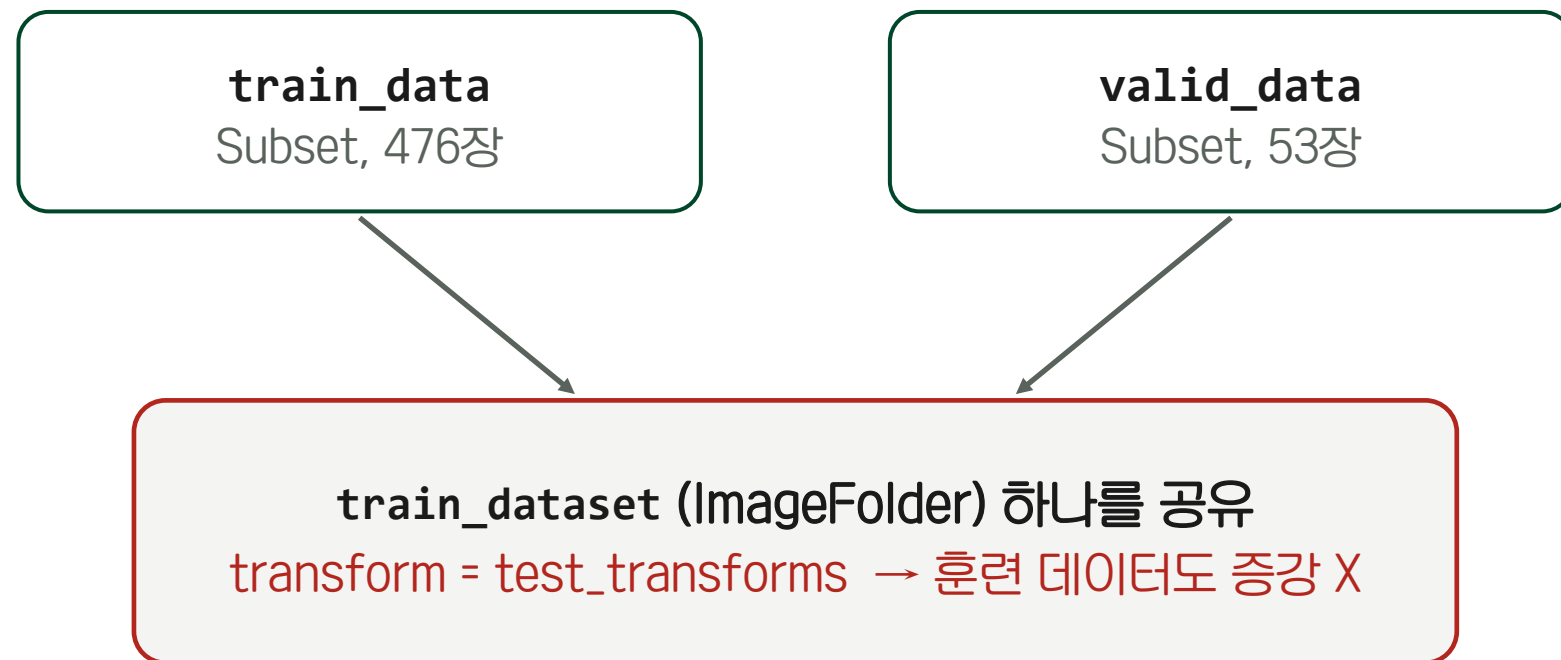
VALID_RATIO = 0.9

```
n_train_examples = int(len(train_dataset)* VALID_RATIO)
n_valid_examples = len(train_dataset) - n_train_examples
train_data, valid_data = data.random_split(
    train_dataset, [n_train_examples, n_valid_examples])
```

- random_split(데이터셋, [개수1, 개수2]) : 무작위로 나눔
- DataLoader로 넘기기 전, Dataset 단계에서 나눠야 함

6.1.3 데이터 준비 ③ 검증 데이터 전처리와 deepcopy

deepcopy 없이 transform만 바꾸면



두 Subset이 같은 ImageFolder를 참조해서 하나를 바꾸면 둘 다 바뀜

#검증 데이터 전처리

```
valid_data = copy.deepcopy(valid_data)
valid_data.dataset.transform = test_transforms
```

deepcopy로 분리한 뒤 적용하면



검증 데이터만 원본과 분리해서 각자 맞는 전처리 적용

```
Number of training examples: 476
Number of validation examples: 53
Number of testing examples: 12
```

random_split의 결과(Subset)는 데이터를 복사하지 않고 원본을 참조만 하기 때문에, 검증용 전처리를 따로 주려면 deepcopy로 먼저 분리해야 함

6.1.3 모델 학습 준비

```
#메모리로 데이터 불러오기
BATCH_SIZE = 32
train_iterator = data.DataLoader(train_data, shuffle=True, batch_size=BATCH_SIZE)
valid_iterator = data.DataLoader(valid_data, batch_size=BATCH_SIZE)
test_iterator = data.DataLoader(test_dataset, batch_size=BATCH_SIZE)

#옵티마이저와 손실함수 정의
optimizer = optim.Adam(model.parameters(), lr =1e-4)
criterion = nn.CrossEntropyLoss()

model = model.to(device)
criterion = criterion.to(device)
```

DataLoader

- 배치 크기만큼 묶어서 꺼내 줌, 훈련 데이터만 shuffle=True (매 에폭 순서를 섞음)
- 배치 수 : 훈련 15개 (마지막 배치 28장), 검증 2개, 테스트 1개

옵티마이저와 손실 함수

- Adam : 파라미터마다 학습률을 자동으로 조절하는 옵티마이저
- CrossEntropyLoss : 소프트맥스 + 음의 로그 가능도라서 모델 출력(로짓)을 그대로 넣음

	배치 크기	학습률 (lr)	에폭	1 에폭 학습 시간
교재	128	1e-7	5	약 7~8분
실습 노트북 (Colab T4 GPU)	32	1e-4	5	약 15~20초 (첫 에폭 1분 36초)

6.1.3 정확도 측정 함수

#모델 정확도 측정 함수

```
def calculate_accuracy(y_pred, y):  
    top_pred = y_pred.argmax(1, keepdim=True)  
    correct = top_pred.eq(y.view_as(top_pred)).sum()  
    acc = correct.float() / y.shape[0]  
    return acc
```

- `argmax(1)` : 클래스 축(dim=1)에서 최댓값의 위치 = 예측
- `keepdim=True` : 결과 모양을 [B] 대신 [B, 1]로 유지
- `view_as(top_pred)` : 정답 y [B]를 [B, 1] 모양으로 맞춤
- `eq, sum` : 같으면 True, True 개수가 맞힌 개수
- 맞힌 개수 ÷ 배치 크기 = 배치 정확도

※ train, evaluate는 이 배치 정확도를 더해 배치 수로 나누기 때문에 마지막 배치가 작으면 (검증 53장 = 32 + 21) 전체 정답 비율과 조금 다를 수 있음

예시 : 배치 4개, 클래스 2개 (Cat=0, Dog=1)

y_pred (로짓)		예측	정답 y	eq																				
<table><tr><td>2.1</td><td>-0.3</td></tr><tr><td>0.2</td><td>1.5</td></tr><tr><td>1.0</td><td>0.9</td></tr><tr><td>-0.5</td><td>0.7</td></tr></table> [4, 2]	2.1	-0.3	0.2	1.5	1.0	0.9	-0.5	0.7	→	<table><tr><td>0</td></tr><tr><td>1</td></tr><tr><td>0</td></tr><tr><td>1</td></tr></table> [4, 1]	0	1	0	1	<table><tr><td>0</td></tr><tr><td>1</td></tr><tr><td>1</td></tr><tr><td>1</td></tr></table> [4]→[4,1]	0	1	1	1	<table><tr><td>True</td></tr><tr><td>True</td></tr><tr><td>False</td></tr><tr><td>True</td></tr></table> [4, 1]	True	True	False	True
2.1	-0.3																							
0.2	1.5																							
1.0	0.9																							
-0.5	0.7																							
0																								
1																								
0																								
1																								
0																								
1																								
1																								
1																								
True																								
True																								
False																								
True																								

$$\text{correct} = 3 \rightarrow \text{acc} = 3 / 4 = 0.75$$

6.1.3 모델 학습 함수 (train)

```
#모델 학습 함수 정의
def train(model, iterator, optimizer, criterion, device):
    epoch_loss = 0
    epoch_acc = 0

    model.train()

    for (x, y) in iterator:
        x = x.to(device)
        y = y.to(device)

        optimizer.zero_grad()

        y_pred, _ = model(x)

        loss = criterion(y_pred, y)
        acc = calculate_accuracy(y_pred, y)

        loss.backward()
        optimizer.step()

        epoch_loss += loss.item()
        epoch_acc += acc.item()
    return epoch_loss / len(iterator), epoch_acc / len(iterator)
```

반환값 : 에폭 평균 손실, 에폭 평균 정확도 (배치별 값을 더해 배치 수로 나눔)

배치 하나에서 일어나는 일

1. `model.train()`

학습 모드 (Dropout 켜짐, BN은 현재 배치 통계)

2. `.to(device)`

데이터를 모델과 같은 장치(GPU)로

3. `optimizer.zero_grad()`

이전 배치의 기울기 초기화 (안 하면 누적)

4. `model(x)`

순전파 (두 번째 반환값 h는 안 써서 _로 받음)

5. `criterion(y_pred, y)`

예측과 정답으로 손실 계산

6. `loss.backward()`

역전파로 기울기 계산

7. `optimizer.step()`

기울기로 가중치 갱신

8. `loss.item()`

텐서에서 숫자만 꺼내 누적

6.1.3 성능 측정 함수

```
#모델 성능 측정 함수
def evaluate(model, iterator, criterion, device):
    epoch_loss = 0
    epoch_acc = 0

    model.eval()
    with torch.no_grad():
        for (x, y) in iterator:
            x = x.to(device)
            y = y.to(device)

            y_pred, _ = model(x)

            loss = criterion(y_pred, y)
            acc = calculate_accuracy(y_pred, y)

            epoch_loss += loss.item()
            epoch_acc += acc.item()
    return epoch_loss / len(iterator), epoch_acc / len(iterator)
```

	train()	evaluate()
모드	model.train()	model.eval()
기울기 계산	0	X (torch.no_grad())
backward, step	0	X (가중치 그대로)
Dropout	켜짐	꺼짐
BatchNorm	현재 배치 통계	저장된 통계

6.1.3 학습 시간 측정 함수

#학습 시간 측정 함수

```
def epoch_time(start_time, end_time):  
    elapsed_time = end_time - start_time  
    elapsed_mins = int(elapsed_time / 60)  
    elapsed_secs = int(elapsed_time - (elapsed_mins * 60))  
    return elapsed_mins, elapsed_secs
```

epoch_time() : 초 → (분, 초)

6.1.3 모델 학습

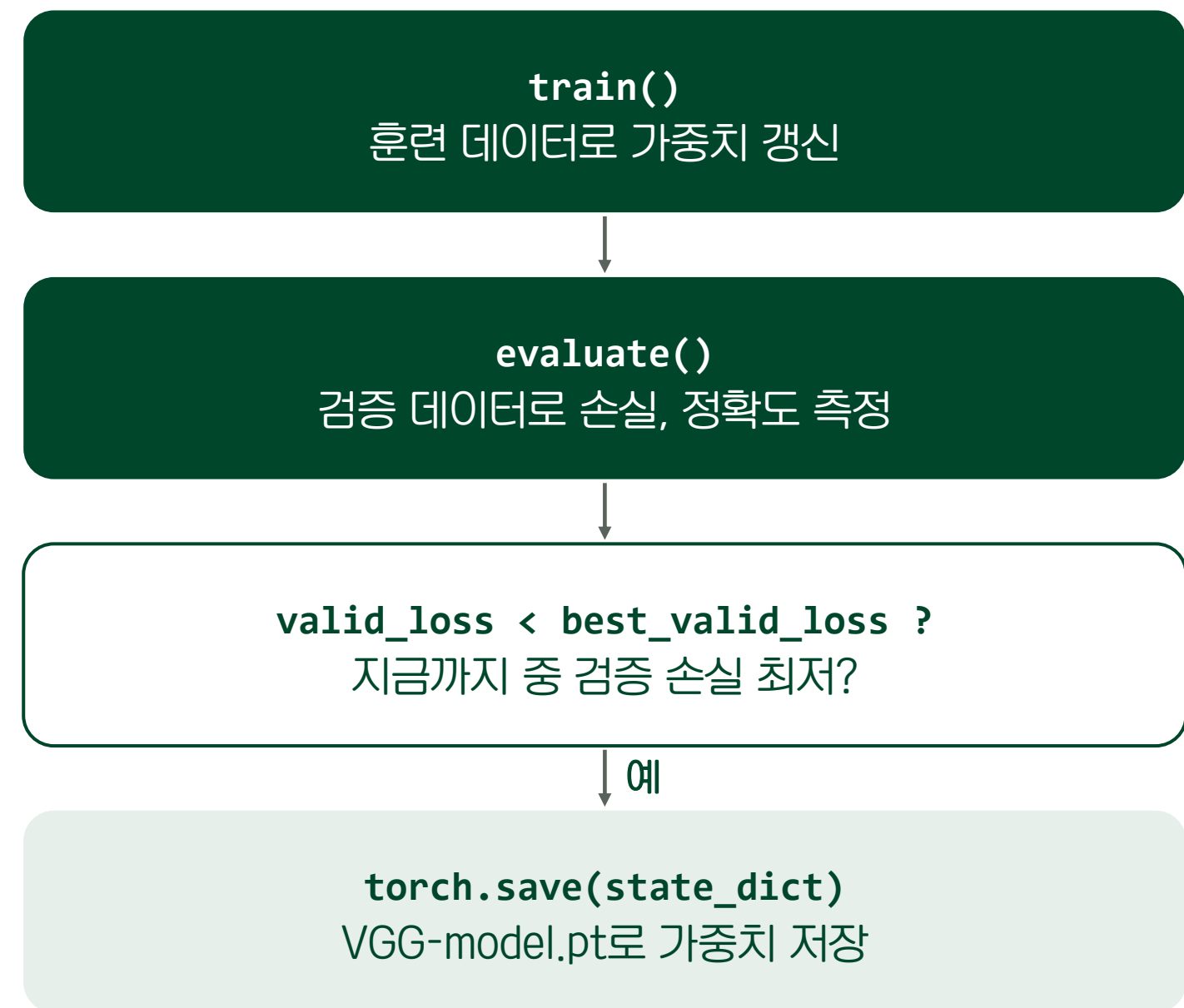
```
#모델 학습
import time
EPOCHS = 5
best_valid_loss = float('inf')
for epoch in range(EPOCHS):
    start_time = time.monotonic()

    train_loss, train_acc = train(model, train_iterator, optimizer, criterion, device)
    valid_loss, valid_acc = evaluate(model, valid_iterator, criterion, device)

    if valid_loss < best_valid_loss:
        best_valid_loss = valid_loss
        torch.save(model.state_dict(), 'VGG-model.pt')
    end_time = time.monotonic()

    print(f'Epoch: {epoch+1:02} | Epoch Time: {epoch_time(start_time, end_time)}')
    print(f'WtTrain Loss: {train_loss:.3f} | Train Acc: {train_acc*100:.2f}%')
    print(f'Wt Val. Loss: {valid_loss:.3f} | Val. Acc: {valid_acc*100:.2f}%')
```

- 검증 손실이 가장 낮을 때의 가중치만 저장해서 가장 좋았던 모델을 남김
- `float('inf')` : 첫 에폭은 무조건 저장되도록 무한대에서 시작
- `time.monotonic()` : 시스템 시각이 바뀌어도 영향 없는 시계

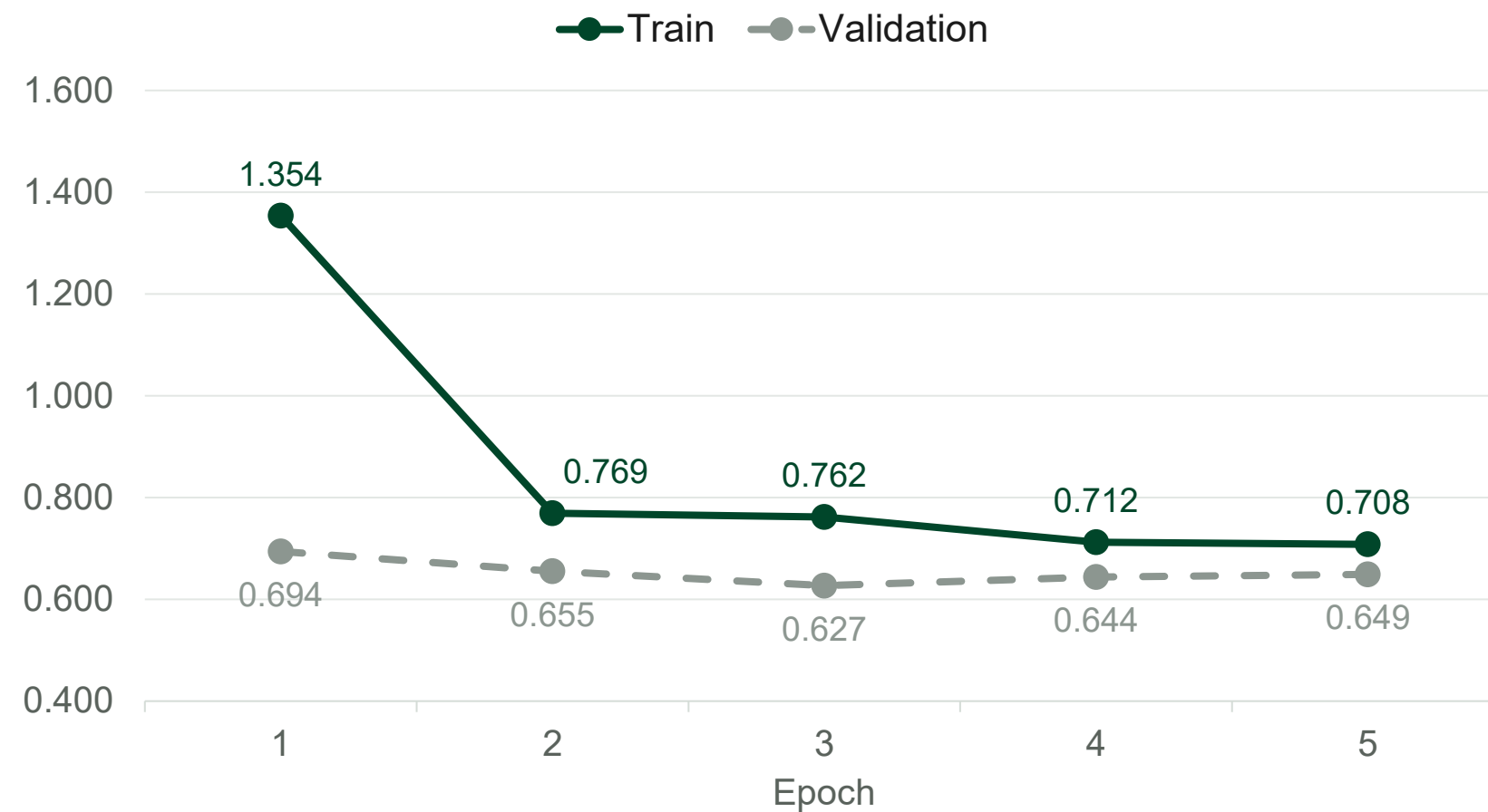


아니오 → 저장하지 않고 다음 에폭으로 (총 5번 반복)

※ 실습 노트북은 `epoch_time()`의 결과를 그대로 출력해서 'Epoch Time: (1, 36)' 형태로 나옴

6.1.3 VGGNet 학습 결과

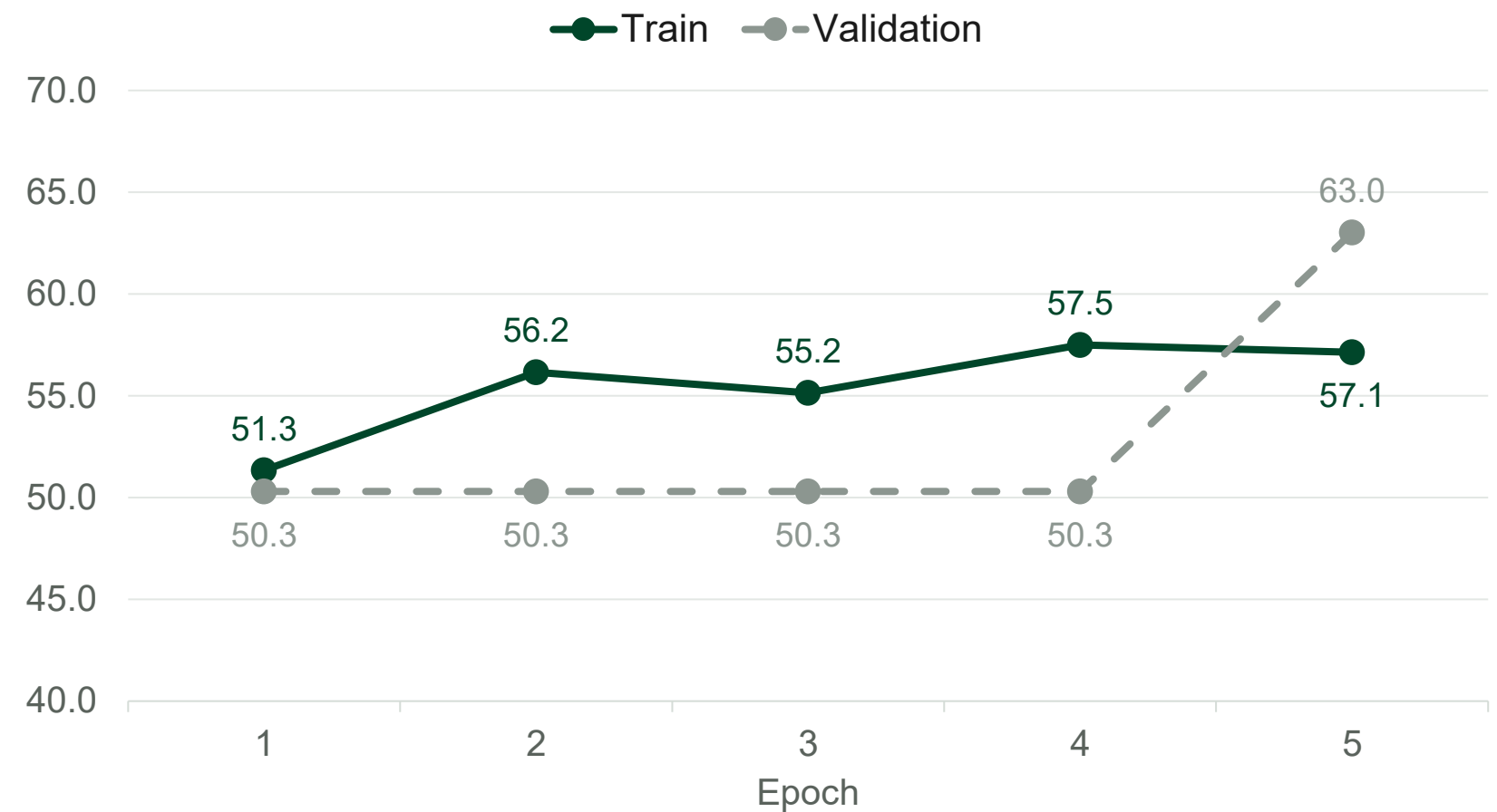
손실 (Loss)



실습 노트북 실행 결과 (Adam, lr=1e-4, batch 32, 5 에폭, Colab T4 GPU)

- 검증 손실이 가장 낮았던 건 **3에폭 (0.627)**이라 이때 가중치를 저장
- 테스트 정확도는 **50.00%** (Test Loss 0.601), 12장 중 6장 정답
- 교재 (lr=1e-7)는 검증 손실이 **0.693**에서 5에폭 내내 변화 없음

정확도 (Accuracy, %)



6.1.3 정확도가 낮은 이유

$0.693 = \ln 2 = -\ln(0.5)$

두 클래스를 50 : 50으로 찍을 때의 교차 엔트로피 손실과
같음. 교재 모델은 거의 학습이 안 된 상태

원인

- 데이터가 너무 적음 : 훈련 476장으로 파라미터 1억 2,878만 개 모델을 처음부터 학습
- 학습 부족 : 5 에폭뿐, 교재는 학습률 1e-7로 가중치가 거의 바뀌지 않음
- 평가가 불안정 : 테스트가 12장뿐이라 1장 차이가 8.3%p
- 실습 : 검증 정확도가 4에폭 동안 50.30%로 고정, 한 클래스로만 예측했을 가능성

개선 방향

- 사전 훈련 모델로 전이 학습 : `models.vgg11_bn(pretrained=True)`
- 데이터 증강/추가, 에폭 늘리기, 학습률 조정 (8장 성능 최적화)
- 테스트 데이터도 충분히 있어야 평가 결과를 믿을 수 있음

	교재	실습 노트북
학습률 (lr)	1e-7	1e-4
최저 Val. Loss	0.693	0.627
Test Loss	0.693	0.601
Test Acc	50.00%	50.00%

6.1.3 테스트 데이터 예측 (get_predictions)

#테스트 데이터셋을 이용한 모델의 예측 확인 함수

```
def get_predictions(model, iterator):
    model.eval()
    images = []
    labels = []
    probs = []

    with torch.no_grad():
        for (x, y) in iterator:
            x = x.to(device)
            y_pred, _ = model(x)
            y_prob = F.softmax(y_pred, dim=-1)
            top_pred = y_prob.argmax(1, keepdim=True)
            images.append(x.cpu())
            labels.append(y.cpu())
            probs.append(y_prob.cpu())
    images = torch.cat(images, dim=0)
    labels = torch.cat(labels, dim=0)
    probs = torch.cat(probs, dim=0)
    return images, labels, probs
```

#테스트 데이터셋을 이용한 모델 성능 측정

```
model.load_state_dict(torch.load('VGG-model.pt'))
test_loss, test_acc = evaluate(model, test_iterator, criterion, device)
print(f'Test Loss: {test_loss:.3f} | Test Acc: {test_acc*100:.2f}%')
```

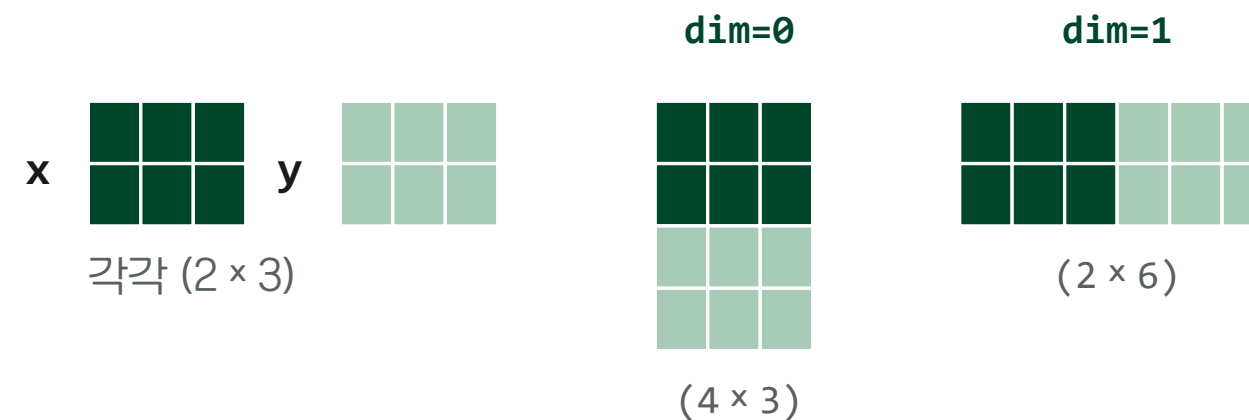
Test Loss: 0.601 | Test Acc: 50.00%

softmax : 로짓 → 확률

각 행(샘플)의 합이 1이 되도록 변환

$[1.2, -0.3] \rightarrow [0.82, 0.18]$

torch.cat : 텐서 이어 붙이기



- 배치별로 모은 리스트를 dim=0(행 방향)으로 연결하면 images [12, 3, 256, 256], labels [12], probs [12, 2]
- .cpu() : GPU 텐서를 CPU로 옮겨 모음 (시각화용)
- top_pred는 계산만 되고 반환되지는 않음

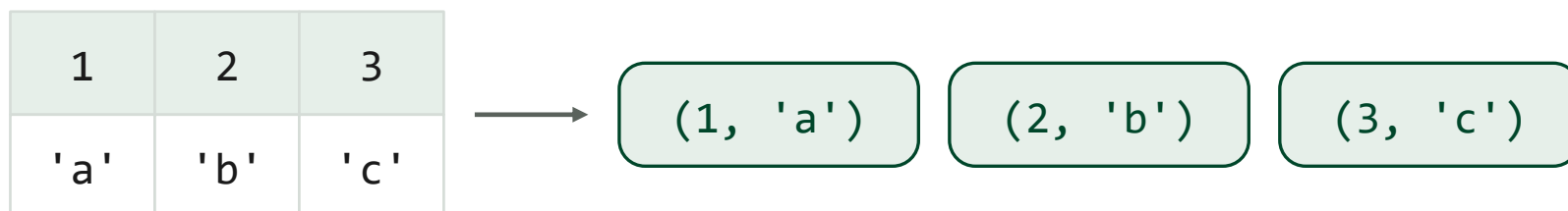
6.1.3 정확하게 예측한 이미지 추출

```
#예측 총에서 정확하게 예측한 것들 추출
images, labels, probs = get_predictions(model, test_iterator)
pred_labels = torch.argmax(probs, 1)
corrects = torch.eq(labels, pred_labels)
correct_examples = []

for image, label, prob, correct in zip(images, labels, probs, corrects):
    if correct:
        correct_examples.append((image, label, prob))

correct_examples.sort(reverse=True, key=lambda x: torch.max(x[2], dim=0).values)
```

zip() : 같은 위치끼리 묶어서 꺼내기



(이미지, 정답, 확률, 정답 여부)를 한 세트씩 꺼내서 맞힌 것만 저장

max vs argmax

max는 가장 큰 값, argmax는 그 위치(인덱스)

```
x = torch.tensor([0.2, 0.9, 0.5])
torch.max(x)           # tensor(0.9000)
torch.argmax(x)        # tensor(1)
torch.max(x, dim=0)    # (values, indices)
```

sort(reverse=True, key=...)

- key : 정렬 기준, 각 샘플의 최대 확률 (모델이 얼마나 확신하는지)
- reverse=True : 내림차순이라 가장 확신 있게 맞힌 이미지부터
- lambda x: ... : 이름 없는 한 줄 함수 (def 없이 바로 정의)

6.1.3 이미지 출력을 위한 전처리



그림 6-20 정규화된 텐서를 그대로 출력하면 색이 깨짐

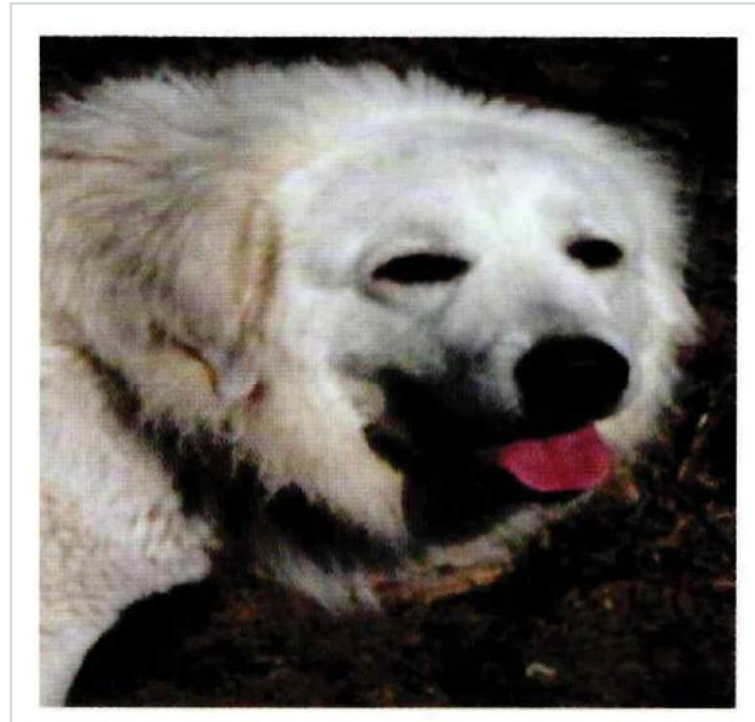


그림 6-21 normalize_image() 적용 후

밑줄(_) 메서드 : in-place 연산

```
x = torch.tensor([1, 2])
y = x.add(10)      # 새 텐서 → x is y : False
y = x.add_(10)     # x 자체를 바꿈 → x is y : True
```

새 메모리를 할당하지 않고 기존 텐서의 값을 직접 바꿈

코드 6-62 normalize_image()

```
#이미지 출력을 위한 전처리
def normalize_image(image):
    image_min = image.min()
    image_max = image.max()
    image.clamp_(min=image_min, max=image_max)
    image.add_(-image_min).div_(image_max - image_min + 1e-5)
    return image
```

- Normalize를 거친 값은 약 $-2.1 \sim 2.6$ 범위인데 imshow는 0~1로 해석해서 색이 깨짐
- min-max 정규화 $(x - \min) / (\max - \min)$ 으로 다시 0~1 범위로
- $1e-5$: 분모가 0이 되는 것 방지
- clamp_ : 값을 [min, max] 범위로 자름

6.1.3 예측 결과 시각화

```
import matplotlib.pyplot as plt
def plot_most_correct(correct, classes, n_images, normalize=True):
    rows = int(np.sqrt(n_images))
    cols = int(np.sqrt(n_images))
    fig = plt.figure(figsize=(25, 20))
    for i in range(rows*cols):
        ax = fig.add_subplot(rows, cols, i+1)
        image, true_label, probs = correct[i]
        image = image.permute(1, 2, 0)
        true_prob = probs[true_label]
        correct_prob, correct_label = torch.max(probs, dim=0)
        true_class = classes[true_label]
        correct_class = classes[correct_label]

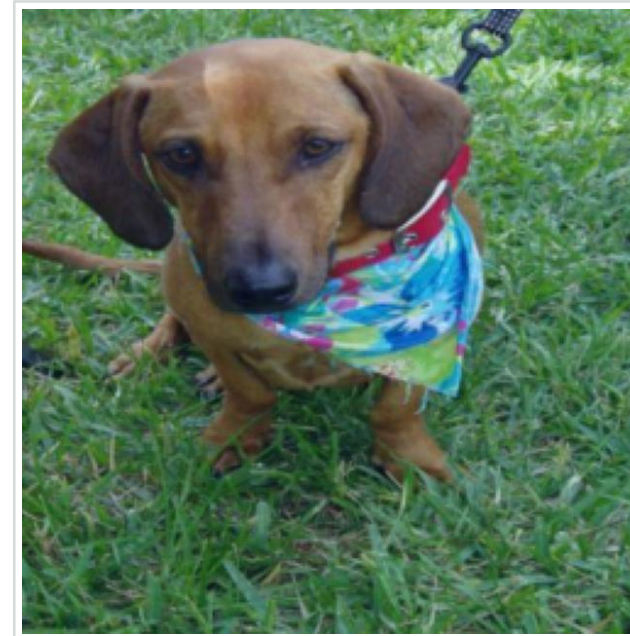
        if normalize:
            image = normalize_image(image)

        ax.imshow(image.cpu().numpy())
        ax.set_title(f'true label: {true_class} ({true_prob: .3f})\n'W
                    f'pred label: {correct_class} ({correct_prob: .3f})')
        ax.axis('off')

    fig.subplots_adjust(hspace=0.4)

classes = test_dataset.classes
N_IMAGES = 5
plot_most_correct(correct_examples, classes, N_IMAGES)
```

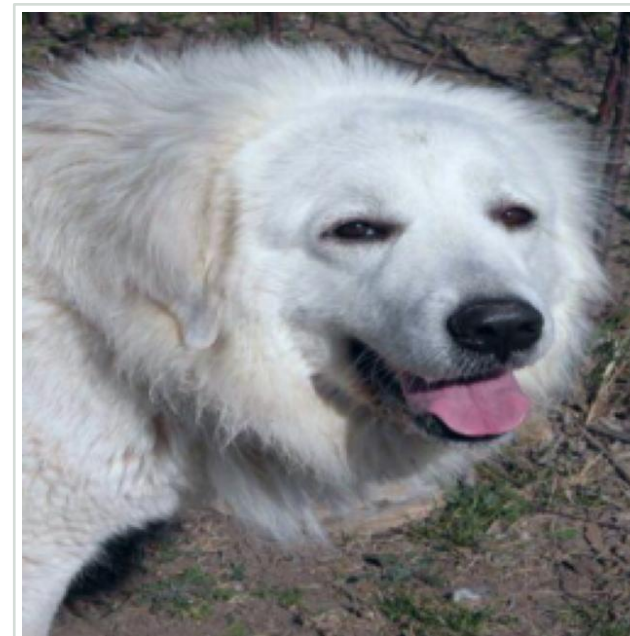
N_IMAGES = 5인데 4장인 이유 : $\text{int}(\sqrt{5}) = 2$ 라서 $2 \times 2 = 4$ 장



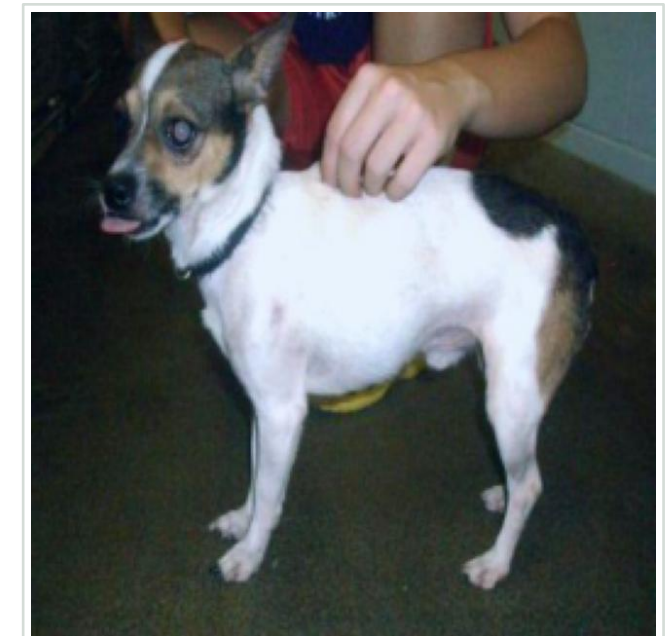
true = pred = Dog (1.000)



true = pred = Dog (0.879)



true = pred = Dog (0.557)



true = pred = Dog (0.545)

맞힌 4장이 모두 Dog이고, 2장은 확률이 0.55 정도로 확신이 약함

6.1.4 GoogLeNet



6.1.4 GoogLeNet 등장 배경

GoogLeNet : 하드웨어 자원을 효율적으로 쓰면서 학습 능력을 높인 깊고 넓은 신경망

깊고 넓은 신경망의 문제점

- 층이 깊고(층 수) 넓을수록(뉴런 수) 인식률은 좋아지지만
 - 과적합, 기울기 소실, 연산량/학습 시간 증가 문제가 같이 생김
- 대용량 데이터를 다루는 대회나 서비스에서는 특히 큰 문제

GoogLeNet (2014)

- 구글 Szegedy 외, “Going Deeper with Convolutions”
- ILSVRC 2014 분류 부문 **1위** (top-5 오류율 6.67%)
- 22층 깊이, 인셉션 모듈 9개 (‘인셉션’ 이라고도 부름)
- 이름은 LeNet에 대한 오마주 (GoogLe + LeNet)

파라미터 수 비교 (torchvision 기준)

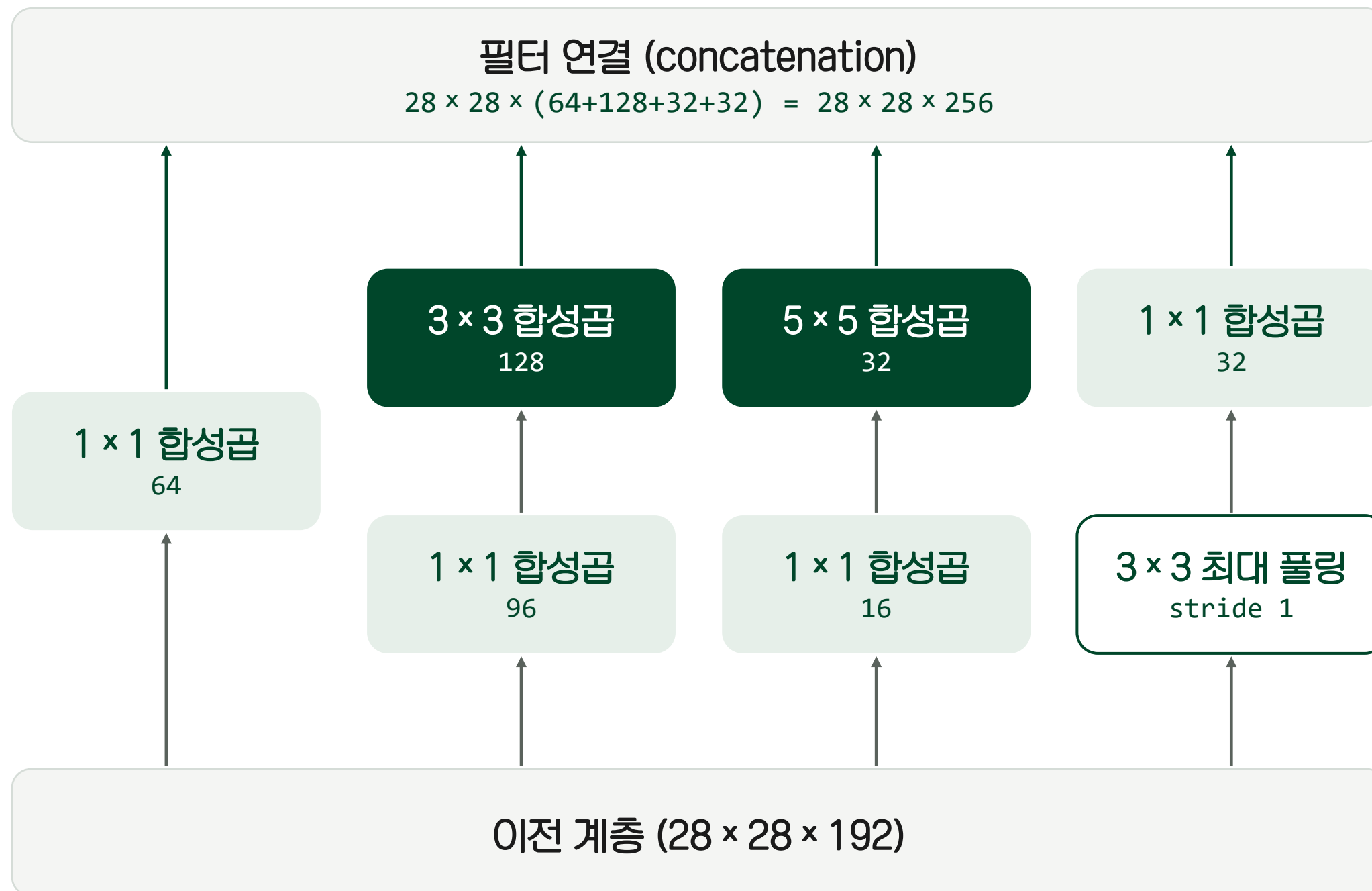
AlexNet (8층)
약 6,100만

VGG16 (16층)
약 1억 3,800만

GoogLeNet (22층)
약 660만

VGG16보다 6층 더 깊은데 파라미터는 약 1/20

6.1.4 인셉션 모듈 (Inception Module)



네 가지 연산

1. 1×1 합성곱
 2. 1×1 합성곱 $\rightarrow$ 3×3 합성곱
 3. 1×1 합성곱 $\rightarrow$ 5×5 합성곱
 4. 3×3 최대 풀링 $\rightarrow$ 1×1 합성곱
- 같은 입력을 여러 크기의 필터로 동시에 봐서 작은 특징과 큰 특징을 한 번에 잡음
 - 결과를 채널 방향으로 이어 붙임 (필터 연결)
 - 높이, 너비가 같아야 연결할 수 있어서 3×3 최대 풀링도 stride 1, padding 1로 크기 유지

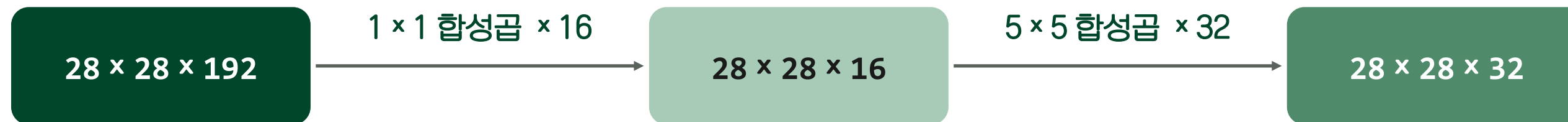
6.1.4 1 × 1 합성곱으로 연산량 줄이기

A. 바로 5 × 5 합성곱



곱셈 약 1억 2,042만 회

B. 1 × 1 합성곱으로 채널을 먼저 줄이기



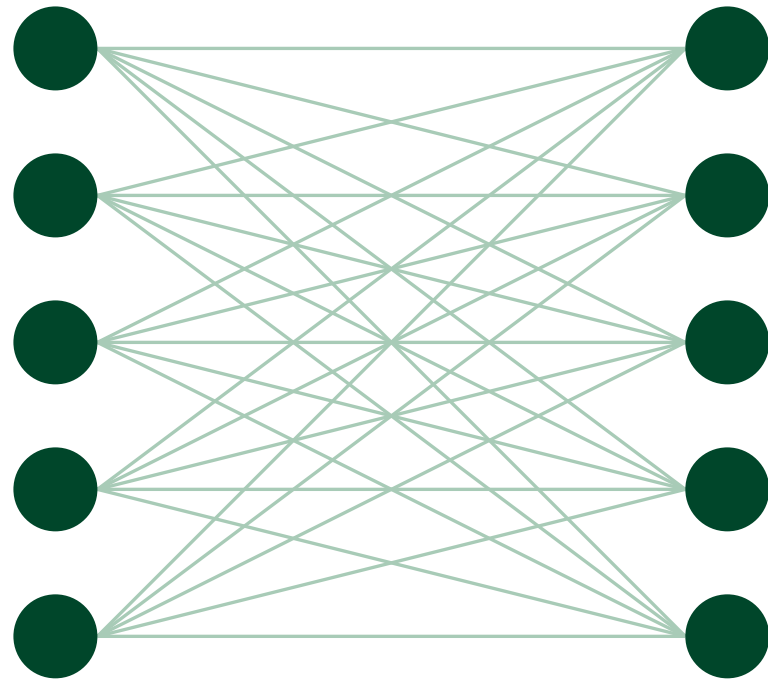
241만 + 1,004만
= 약 1,244만 회



- 1 × 1 합성곱 : 높이와 너비는 그대로 두고 채널 수만 바꾸는 연산 (채널 축소)
- 곱셈 수 = 출력 크기(28 × 28 × 출력 채널) × 필터 크기 × 입력 채널이라 5 × 5의 입력 채널을 192에서 16으로 줄인 셈
- 출력 모양은 같은데 연산량은 약 1/10이고 ReLU도 붙어 비선형성이 늘어남. 남은 자원으로 더 깊고 넓게 쌓음

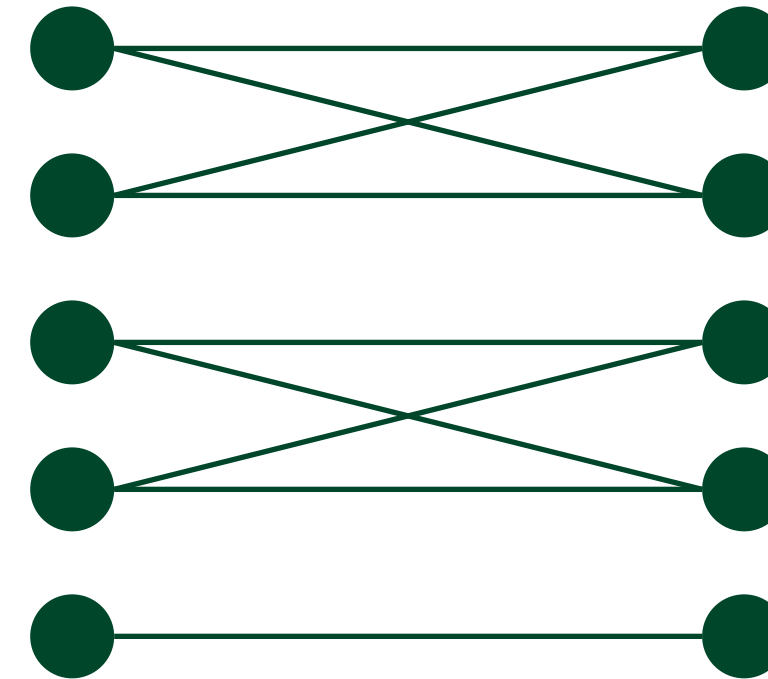
6.1.4 희소 연결 (Sparse Connectivity)

밀집 연결 (dense)



모든 노드를 뺑뺑하게 연결
기존 CNN은 합성곱 / 풀링 / 완전연결층 밀집 연결

희소 연결 (sparse)



관련성이 높은 노드끼리만 연결해서 연산량과 과적합이 줄어듦

GoogLeNet은 **희소 연결**을 씀. 크기가 다른 필터 묶음(인셉션 모듈)이 관련 있는 특징끼리 나눠 처리
계산은 GPU가 빠르게 처리하는 합성곱 연산을 그대로 써서 희소 구조의 장점과 계산 효율을 둘 다 챙김

6.1.4 GoogLeNet 아키텍처의 장점

	AlexNet (2012)	VGG16 (2014)	GoogLeNet (2014)
층 수	8	16	22
파라미터	약 6,100만	약 1억 3,800만	약 660만
ILSVRC 대회 top-5 오류율	15.3% (2012 1위)	7.3% (2014 2위)	6.67% (2014 1위)
핵심 아이디어	ReLU, 드롭아웃, GPU	3 × 3 필터 반복	인셉션, 1 × 1 병목

※ 대회 오류율은 여러 모델을 앙상블한 제출 결과, 파라미터는 torchvision 단일 모델 기준

깊이 + 너비

여러 크기 필터를 병렬로
깊고 넓게 쌓아 표현력 향상

1 × 1 병목

1 × 1로 채널부터 줄임
곱셈-덧셈 약 15억 회
(VGG16은 약 155억 회)

평균 풀링

큰 FC층 대신 평균 풀링
파라미터가 크게 줄어듦
(VGG16은 89%가 FC층)

보조 분류기

중간층에서도 손실 계산
기울기 소실 완화
(학습 때만 사용)

정리

VGGNet

3 × 3 필터를 반복해서 깊게 쌓기

- 3 × 3 합성곱 + 2 × 2 풀링을 반복하는 단순한 구조
- 깊이가 성능을 높인다는 것을 보여 줌 (11 → 19층)
- 파라미터 대부분이 FC층에 있어 무거움 (약 1.38억)
- 실습 : 데이터가 적어서 처음부터 학습하기엔 한계, 전이 학습이 필요

16층, 파라미터 1.38억, top-5 7.3%

GoogLeNet

깊고 넓지만 파라미터는 적게

- 인셉션 모듈 : 여러 크기 필터를 병렬로 써서 너비 확장
- 1 × 1 병목과 평균 풀링으로 연산량, 파라미터 절감
- 22층인데 파라미터는 약 660만 개
- 보조 분류기로 깊은 층까지 학습

22층, 파라미터 약 660만, top-5 6.67%



5주차 질문

6장: 합성곱 신경망 2

질문1: 서영

Q. Pooling을 반복하면 계산량을 줄이고 더 넓은 영역의 특징을 볼 수 있지만 동시에 정확한 위치 정보는 손실됩니다. 이미지 분류에서는 이러한 정보 손실이 큰 문제가 되지 않을 수 있지만 객체 탐지나 segmentation에서는 어떤 문제가 발생할까요?

질문2: 조수진

Q. 질문 2: GoogLeNet에서는 인셉션 모듈을 사용해서 과적합과 기울기 소멸 문제를 해결한다고 설명되어 있는데, 여러 크기의 합성곱을 병렬로 사용하는 인셉션 구조가 기울기 소멸 문제를 구체적으로 어떻게 완화하는지 궁금합니다. 인셉션 모듈 자체가 기울기 소멸 문제를 해결하는 것인지, 아니면 다른 구조적 요소가 영향을 주는 것인지 궁금합니다.

Q. 논의: 코드 6-32의 torchsummary 결과를 보면 합성곱층들의 파라미터 수는 많아야 수십만 단위인데, 완전연결층 하나의 파라미터 수는 3700만 개가 넘어서 전체 파라미터의 대부분을 차지합니다. 책에서는 AlexNet의 파라미터가 총 6000만 개이고 데이터가 부족하면 과적합이 발생한다고만 설명하는데, 파라미터가 합성곱층이 아니라 완전연결층에 이렇게 몰리는 이유가 뭔지, 그리고 이 구조적 특징이 과적합과 어떻게 연결되는지 같이 추론해보고 싶습니다.

감사합니다!

